

DevSecOps 3-3-DAST

Task 1 - Introduction

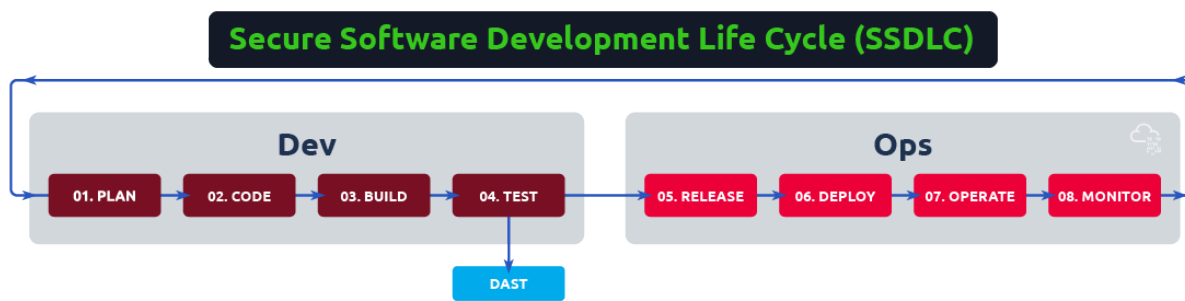
One of the many ways to test an application for vulnerabilities is to take a running instance and attack it just like an outsider attacker would. This process is known as Dynamic Application Security Testing (DAST).

Task 2 - Dynamic Application Security Testing (DAST)

Dynamic Application Security Testing (DAST) is the process of testing a running instance of a web application for weaknesses and vulnerabilities. It focuses on a black-box testing approach where vulnerabilities are found just like a regular attacker would find them. Simply put, DAST identifies vulnerabilities by trying to exploit them, either manually or through automated tools. There are two ways in which DAST can be performed:

- **Manual DAST:** A security engineer will manually perform tests against an application to check for vulnerabilities.
- **Automatic DAST:** An automated tool will scan the web application for vulnerabilities.

Both processes are complementary and can be used at different stages of the **Software Development Lifecycle (SDLC)**. Manual DAST will help us find weaknesses that an automated tool won't easily spot as they don't understand the application from a functional point of view. DAST in the SDLC:



While every scenario is different, it is common to use automated DAST during the test phases. Automated DAST tools will often be optimised for speed so developers can obtain feedback quickly, unlike a traditional web application vulnerability scanner, which could take several hours to run.

Manual DAST, on the other hand, is often run periodically to avoid slowing down the development process as a whole. It is usual to run manual scans weekly while applications are being developed, but this greatly depends on each project. A more intense scanning profile can be used in these manual scans to identify even more vulnerabilities.

DAST Pros and Cons

Pros:

- Finds vulnerabilities during runtime. This will include vulnerabilities that are specific to the deployment process, which can't be seen by analysing code alone.
- DAST will find vulnerabilities like HTTP Request Smuggling, cache poisoning, and parameter pollution that won't be found using SAST.
- DAST tools don't care about the programming language in use by your application. Since DAST is black-box testing, all apps are treated in a language-agnostic way.
- Reduced number of false positives as compared to SAST.
- DAST tools might be able to find some business logic flaws. The effectivity will depend on the tool, and shouldn't be taken as a replacement for manual testing.

Cons:

- Code coverage isn't the best. DAST may not cover specific situations that will only be triggered by specific use cases in your application.
- Some vulnerabilities may be harder to find using DAST, as compared to static code analysis techniques.
- Some apps are difficult to be crawled. Modern applications heavily rely on javascript processing for the client-side. This makes it harder for DAST tools to traverse them.
- DAST scanners won't be able to tell you how to remediate some vulnerabilities in detail, since they have no idea of the underlying code.
- Some types of scans might take lots of time to finish.
- You need a running application to do the testing.

DAST tool will perform at least the two following tasks against the target website:

- **Spidering/Crawling:** The tool will navigate through the web app, trying to
- map the application and identify a list of pages and parameters that can be attacked.

- **Vulnerability Scanning:** The tool will try to launch attack payloads against the identified pages and parameters. The user can typically customise the type of attacks to include only the ones relevant to the target application.

We will use ZAP proxy as our DAST tool for demonstration purposes in this room. Consider, however, that many DAST tools used in enterprise setups are paid and quite expensive but will use the same base principles as any other web application vulnerability scanner. The main difference among tools will be in their vulnerability detection capabilities and how many additional features they add to their product that are useful in enterprises, like dashboards, ticketing system integration and many others.

Zed Attack Proxy (ZAP) is a free and open-source web application security scanner. It is a powerful tool that penetration testers and security professionals can use to test the security of web applications. Zero-hour auto purge (ZAP) in Microsoft Defender for Office 365 retroactively detects and neutralises malicious phishing, spam, or malware messages that were delivered to cloud mailboxes.

ZAP Proxy (commonly called OWASP ZAP) is a free, open-source web application security testing tool developed by the OWASP. It acts as an intercepting proxy between your browser and a web application, allowing you to inspect, modify, and test HTTP/HTTPS traffic for security vulnerabilities.

What ZAP is used for

Finding security vulnerabilities in web applications

Testing APIs (REST, GraphQL, SOAP)

Identifying issues such as:

SQL Injection

Cross-Site Scripting (XSS)

Broken authentication

Security misconfigurations

Sensitive information exposure

Automated vulnerability scanning

Manual penetration testing

Key features

- **Intercepting Proxy** – View and modify web traffic.
- **Spider/Crawler** – Discover website pages automatically.

- **Active Scanner** – Test for vulnerabilities.
- **Passive Scanner** – Analyze traffic without sending attack payloads.
- **Fuzzer** – Send multiple test inputs to parameters.
- **API Testing** – Import OpenAPI/Swagger specifications.
- **Automation Support** – CI/CD integration for DevSecOps pipelines.

Answer the questions below

| Is DAST a replacement for SAST or SCA?

Nay

| What is the process of mapping an application's surface and parameters usually called?

Spidering/Crawling

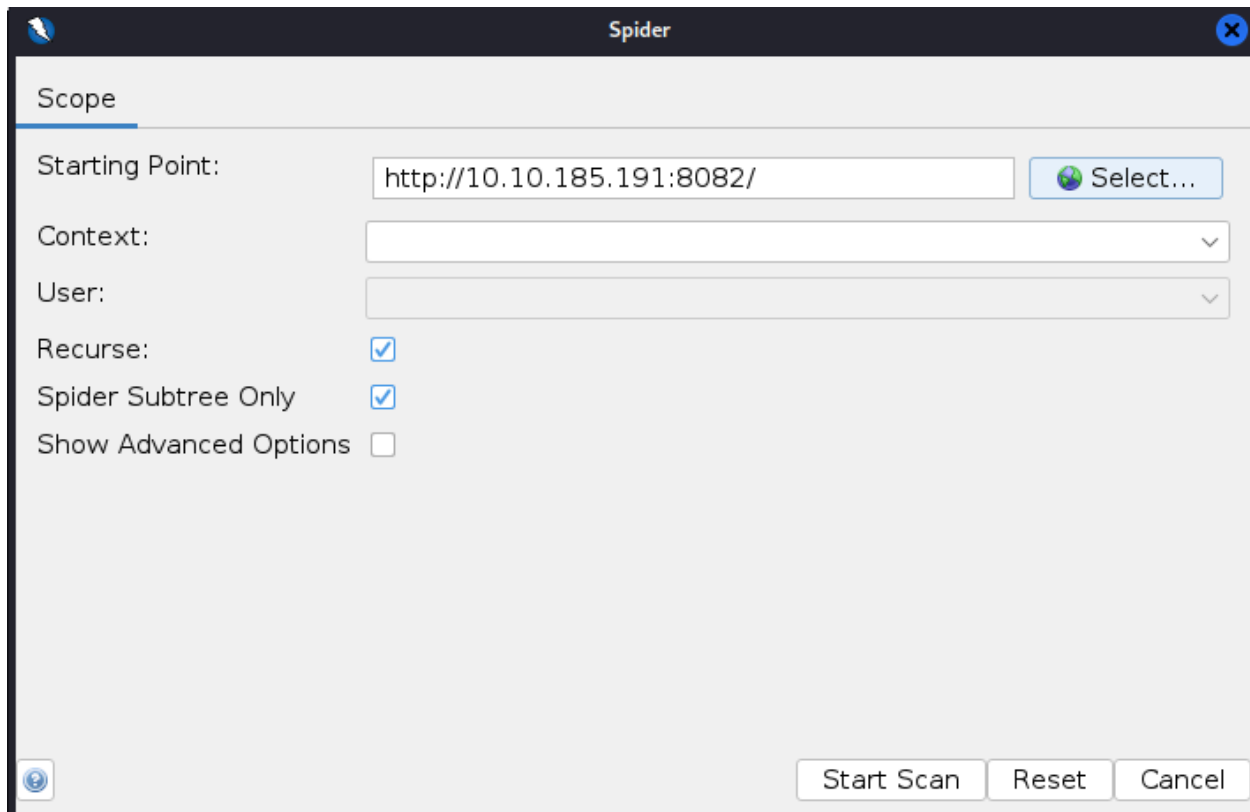
| Does DAST check the code of an application for vulnerabilities?

Nay

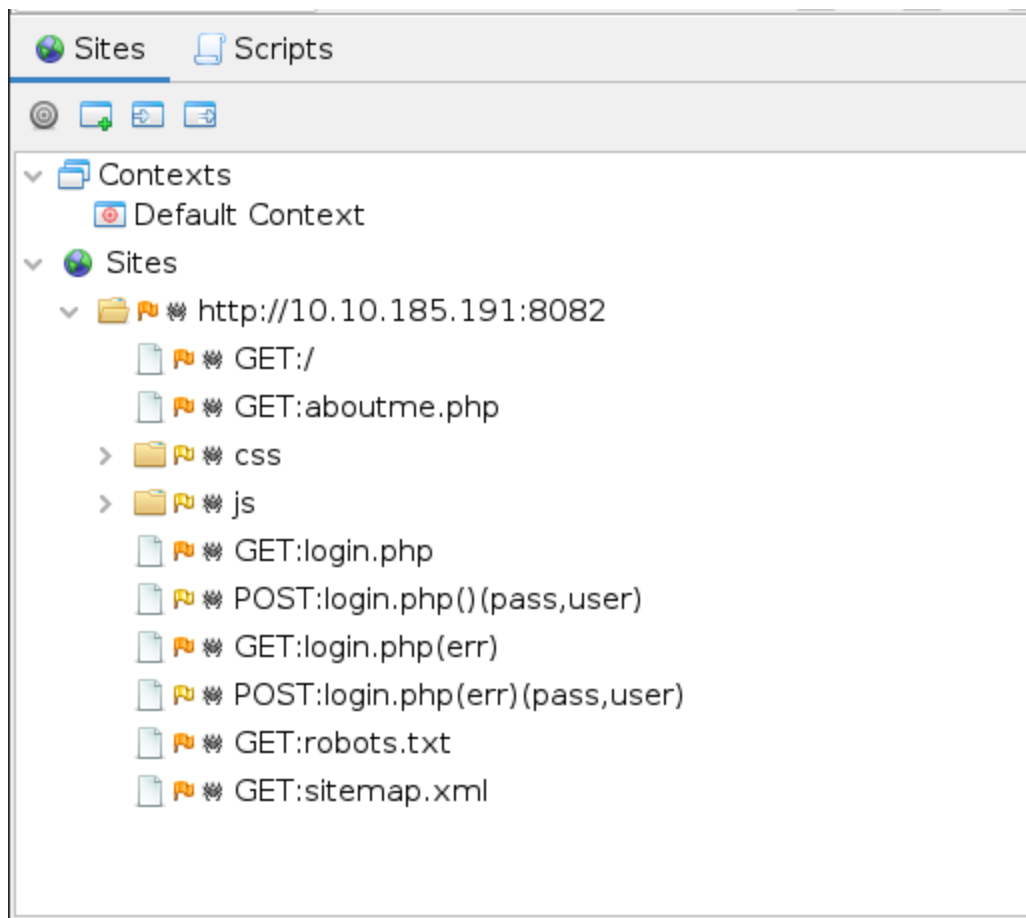
Task 3 - Spiders and Crawlers

Spidering an app with ZAP: ZAP provides a spidering module that we can use to that end. A **Spider** will navigate to a starting page you provide and fundamentally explore the website by following all the links it can detect. To spider a website, we can go to **Tools → Spider**. This will show us the Spider dialogue, where we only need to specify a website as our starting point.

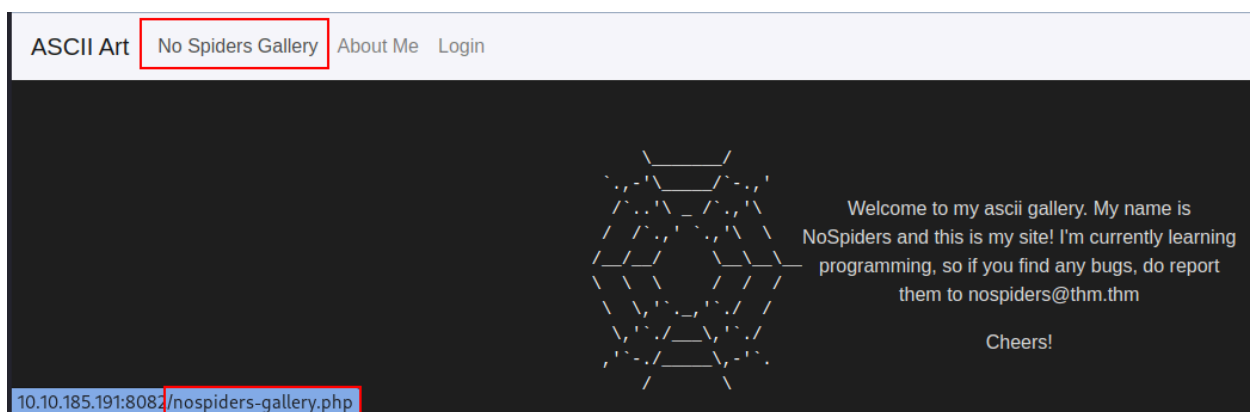
- **Recurse:** Spider the website for links recursively.
- **Spider Subtree Only:** Limit the spidering to subfolders of the Starting Point URL.
- **Show Advanced Options:** Enable the Advanced tab, where you can specify additional parameters to tune your spidering.



Point to http://MACHINE_IP:8082/(opens in new tab) and press Start Scan. After a while, you should see all the URLs that the spider found on the Sites tab:



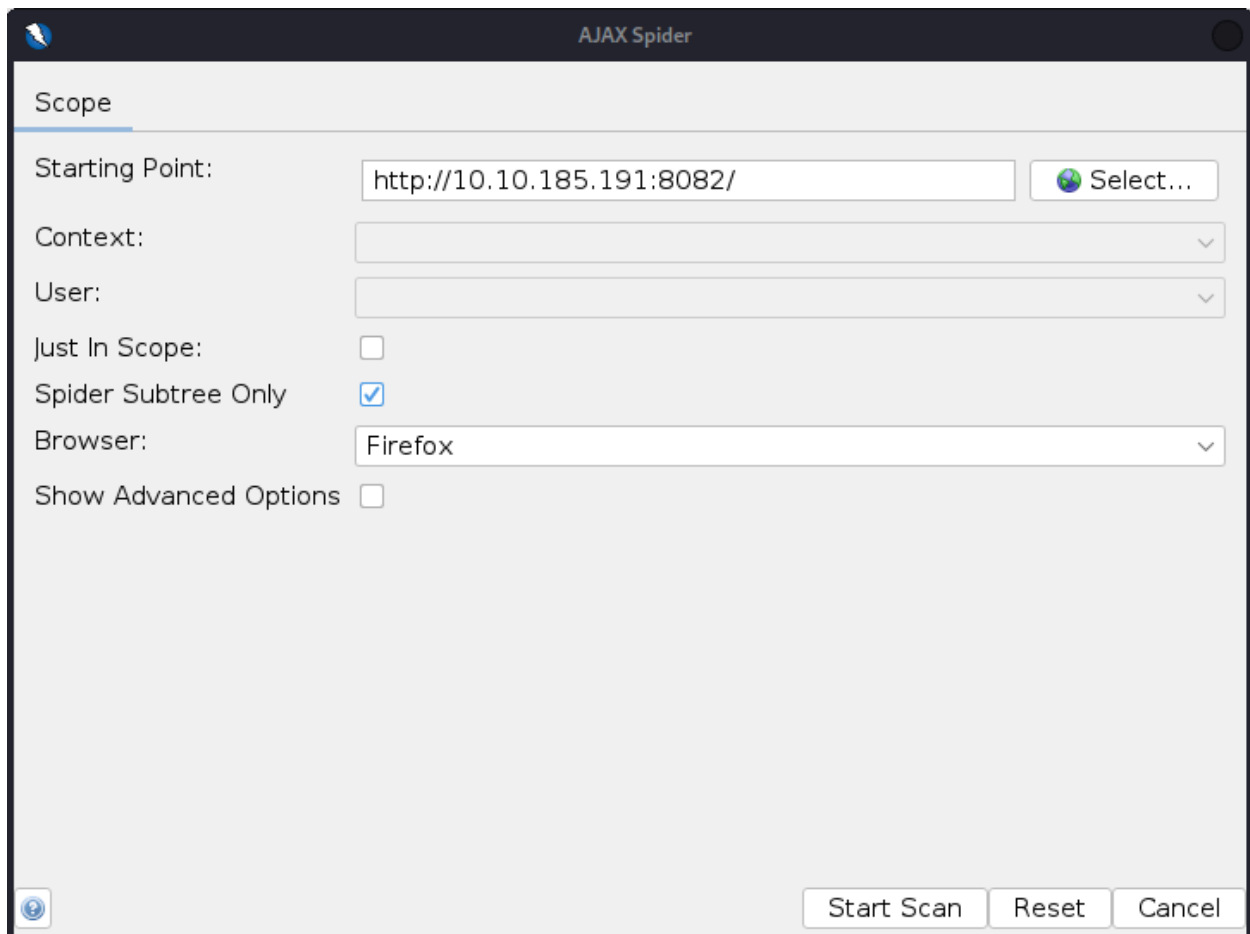
You should now see the Sites list populated with all URLs found during spidering. Any item outside your starting page's URL is considered out-of-scope and won't be added to the list. You might notice that if you navigate manually to http://MACHINE_IP:8082/ (opens in new tab), you should see a link in the menu that points to </nospiders-gallery.php>, but if you expand the sites list on ZAP, you won't see this URL.



This happens because this particular link is not directly embedded in the page's HTML code but is generated on-the-fly by using javascript on your browser (as many modern applications do). Since the regular spider in ZAP doesn't have a javascript engine, it has no way of finding that link.

To overcome the limitations of ZAP's regular spider, ZAP can leverage a real browser like Firefox or Chrome to process any scripts attached to the website and retrieve the resulting HTML code. Instead of parsing links directly from HTTP responses, it will do so from the result of a browser processing the website. This can be achieved by using the **AJAX spider**. Leveraging a real browser is highly advantageous, as ZAP doesn't have to deal with all of the intricacies of current javascript engines but rather delegates this task to a browser, guaranteeing that the result will match what a user will see if they use the same browser.

Using the AJAX spider is a similar process to the regular spider. Just go to **Tools - > AJAX Spider**, configure a URL as a starting point and check the options you see fit. The only new option you have is the Browser selection, where we will choose Firefox this time. The list of browsers also has headless options, which will run the selected browser without showing you its graphical interface, but for now, let's stick to regular Firefox, as it makes it easier to understand how the AJAX spider works.



You should see Firefox popping up and navigating the website for a bit. When the AJAX spider is done, it will populate the Sites in ZAP, just as the regular spider. Since the AJAX spider leverages the power of Firefox to get better results, you should now see `/nospiders-gallery.php` in your Sites list in ZAP.

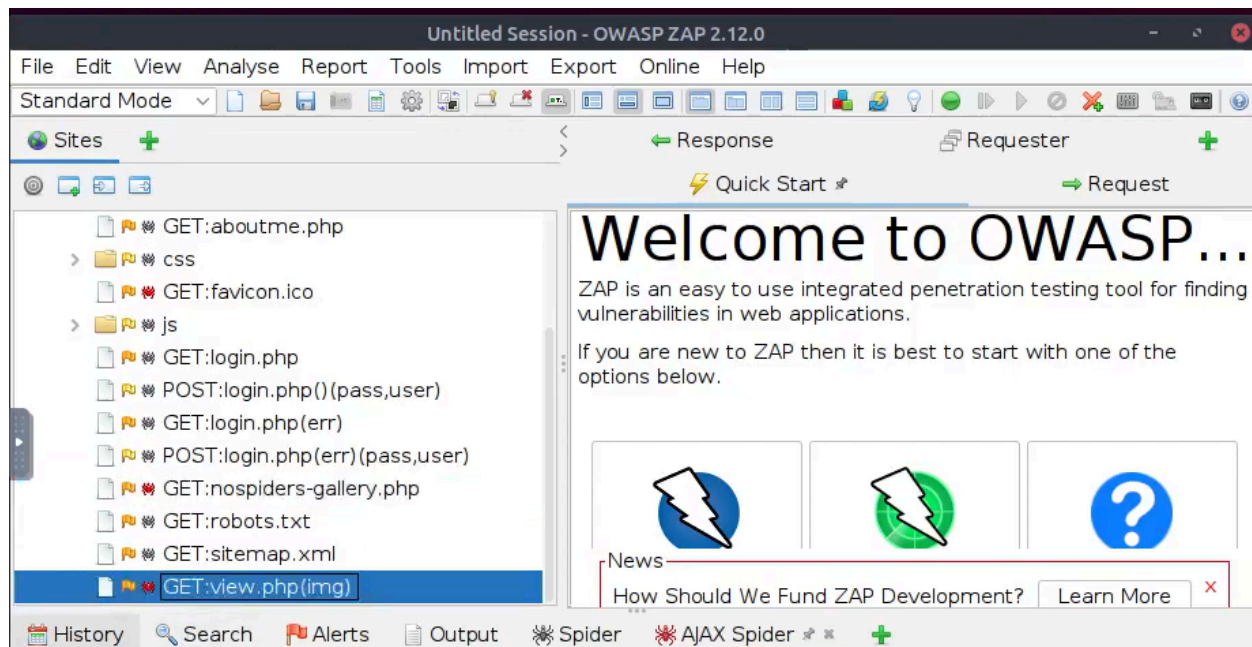
Answer the questions below

ZAP can run an AJAX spider by using browsers without a Graphical User Interface(GUI). What are these browsers called?

Headless

Analysing the Sites tab, what HTTP parameters can be passed to login.php using the POST method?

pass, user

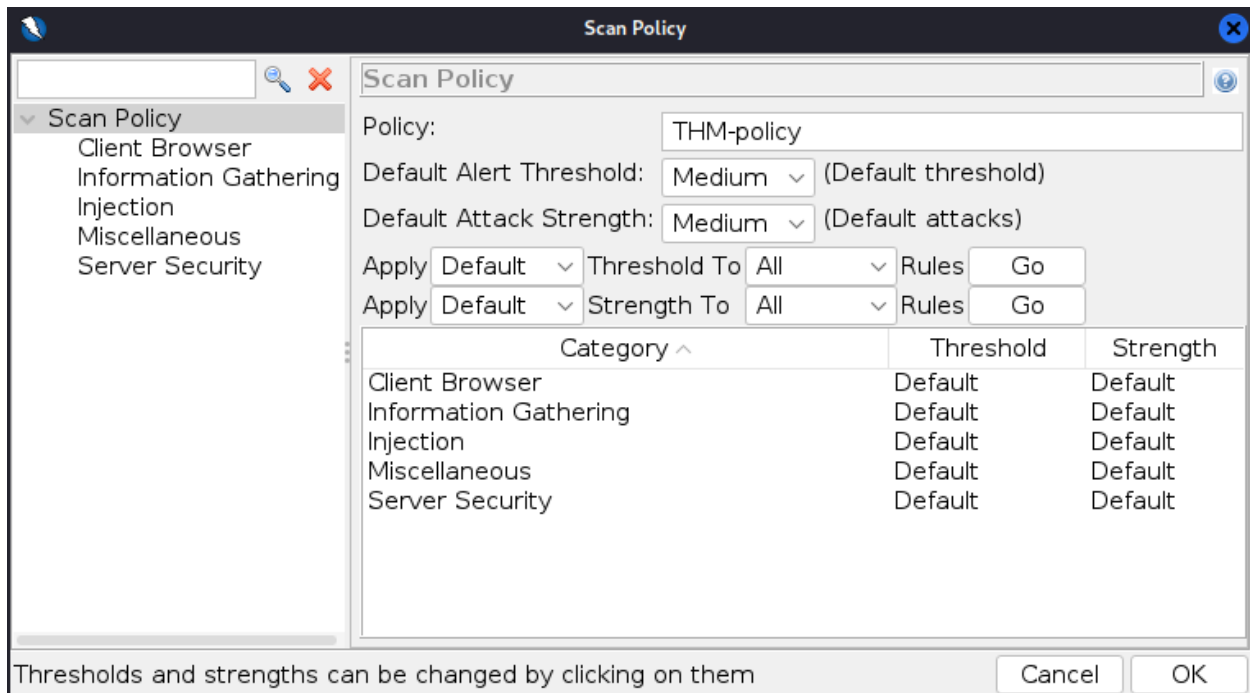


What other .php resource, besides nospiders-gallery.php was found by the AJAX spider but not by the regular spider?

/view.php

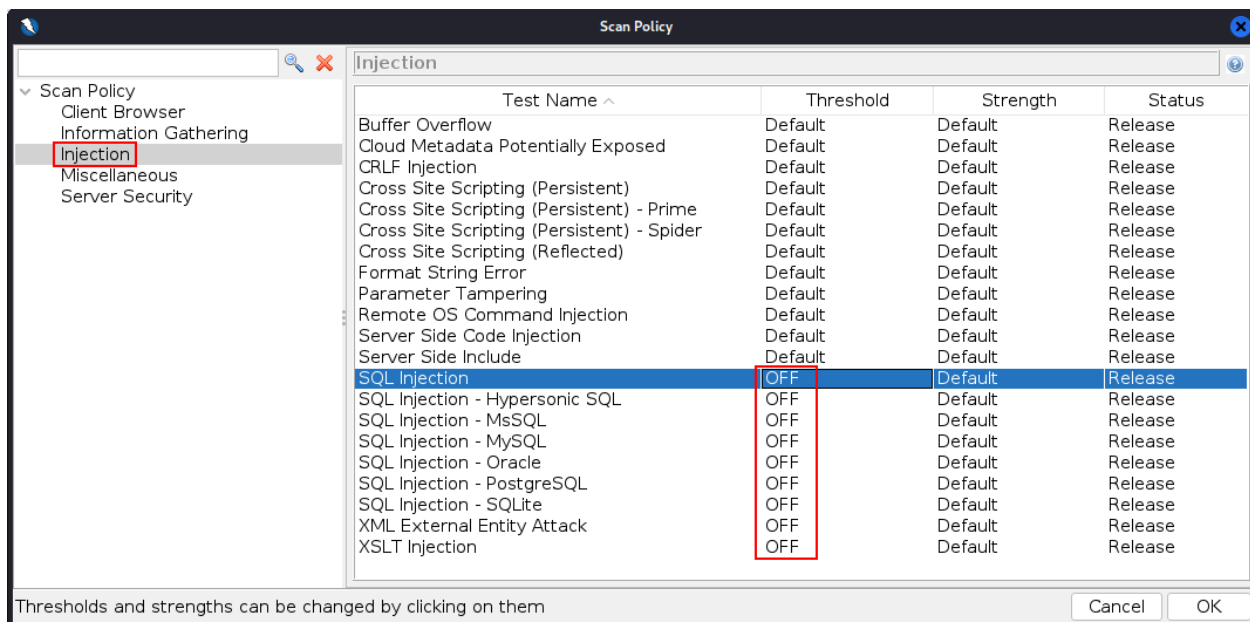
Task 4 - Scanning for Vulnerabilities

To create or modify scanning policies in ZAP, go to **Analyse → Scan Policy Manager**. In there, let's click the Add button to create a new one:

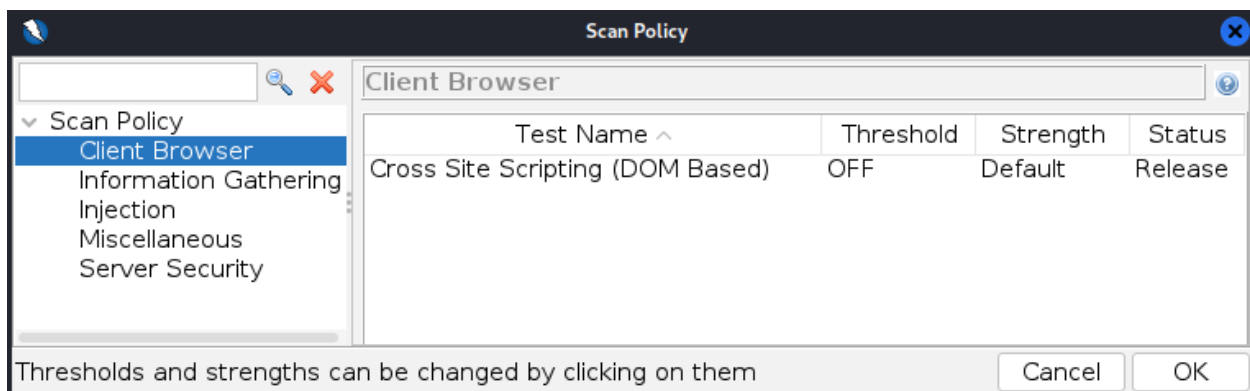


From here, you can control the types of tests to be run as part of your policy. For each category, you will have two parameters to configure:

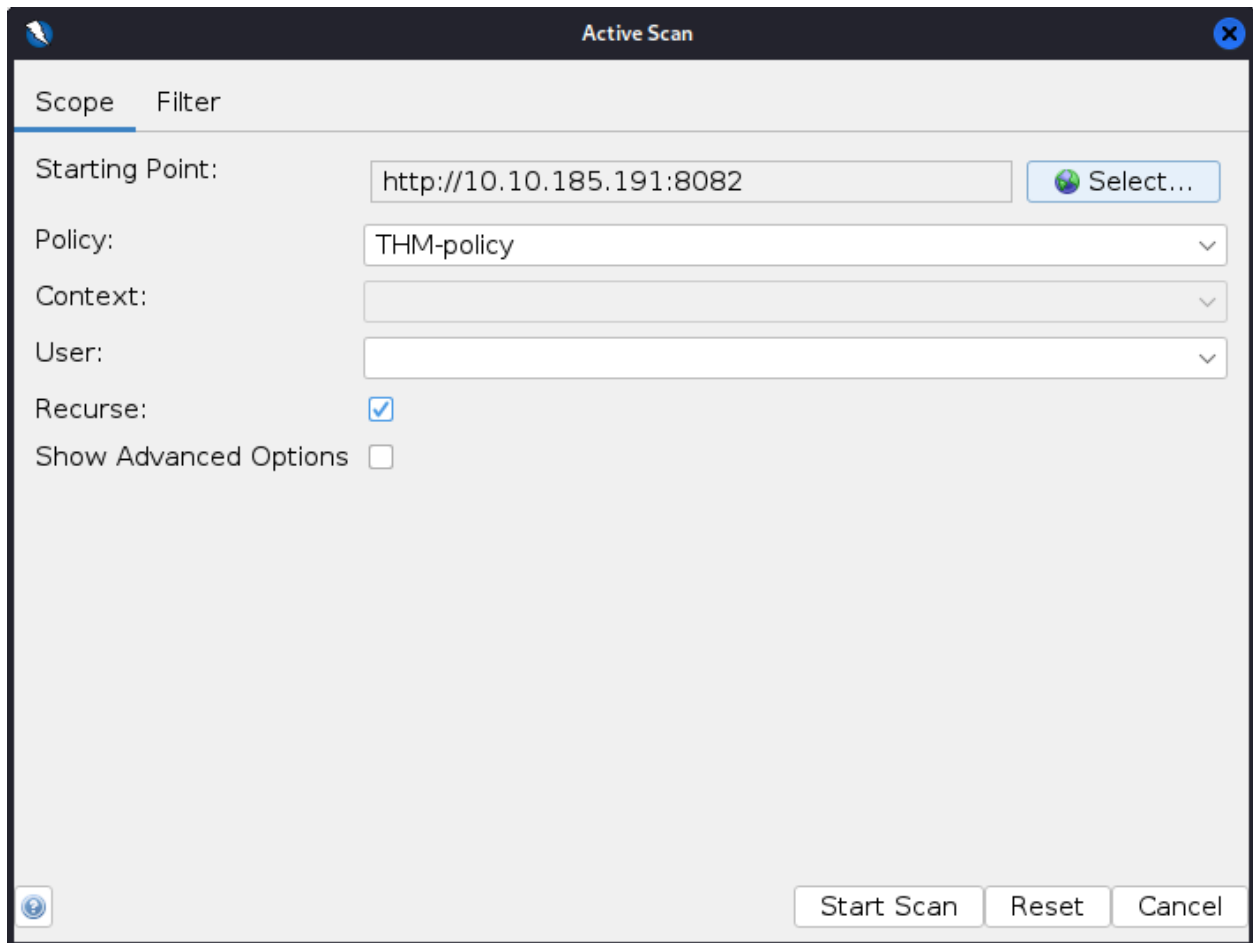
- **Threshold:** This parameter controls the verbosity of each category. A lower threshold means ZAP is more likely to report a vulnerability based on little information. This would get you more findings at the risk of having ZAP report vulnerabilities that may not be present (False positives). A higher threshold will indicate ZAP to only report findings with a high degree of certainty. This will get you fewer results, but those are more likely confirmed. The risk of using higher thresholds is that ZAP may not report some vulnerabilities that might actually be present (False negatives). You can also set the threshold to OFF to disable a category.
- **Strength:** This parameter controls how many tests are run for each category. Higher strength levels are likely to report additional findings at the expense of increasing scanning times significantly.



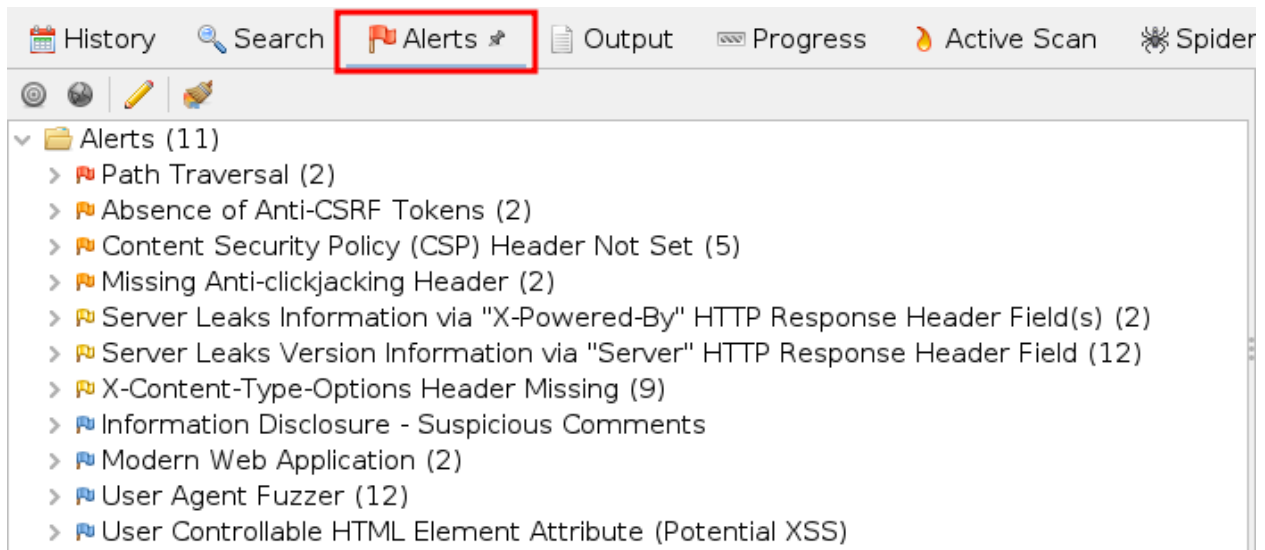
Make sure to also disable the Cross Site Scripting (DOM Based) tests this time, as they take lots of resources and are likely to slow down your scans for this room:



Running our first scan: Now that we have a customised scanning policy, let's run a vulnerability scan by going to **Tools → Active Scan** in the menu. Here we can select a starting point from the previously spidered resources and our newly created scan policy.



After a while, we should see some results in the **Alerts** section. You can get a description for each of the findings that should indicate what was found in general terms. You can also check the specific request and the corresponding response that ZAP used to detect the vulnerability.



This allows you to review the findings to ensure they are all correct. Should you think any of them is a false positive, you can mark it as such by right-clicking the finding and selecting Mark as False Positive.

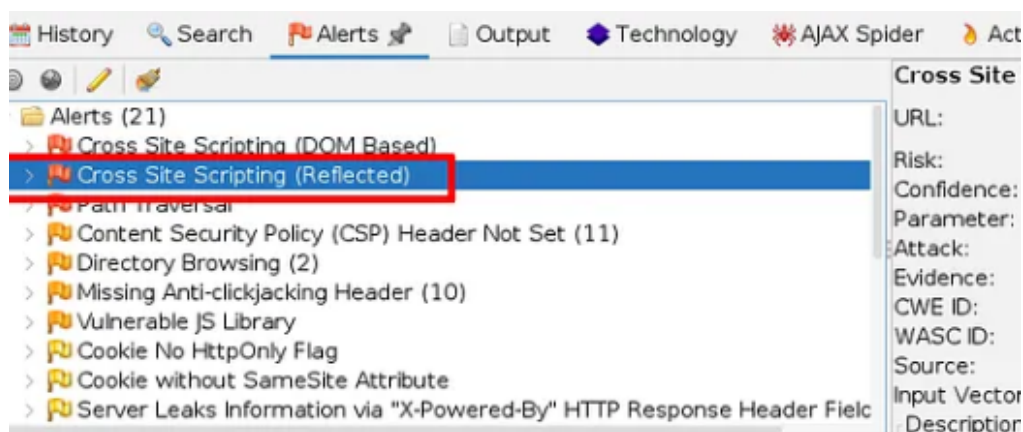
Answer the questions below

Will disabling some test categories help speed up the scanning phase?

Yea

There should be two high-risk alerts in your scan results. One is Path Traversal. What's the name of the other one?

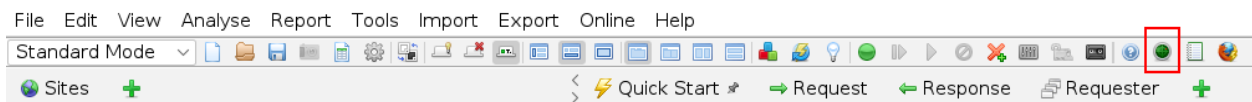
Cross Site Scripting (Reflected)



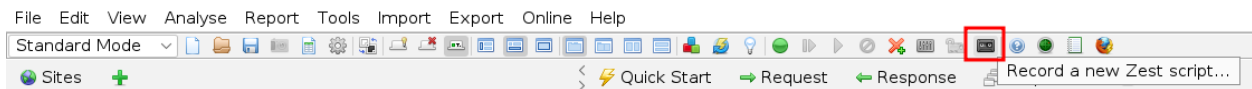
Task 5 - Authenticated Scans

Dealing with logins: Since ZAP knows not about these credentials, it won't be able to check anything that requires authentication. We must teach ZAP how to authenticate and ensure its session is active to solve this problem. To do so, we will record the authentication process into a ZEST script so that ZAP can replicate the process during scans. For now, think of a ZEST script as a way to save and reproduce a series of requests.

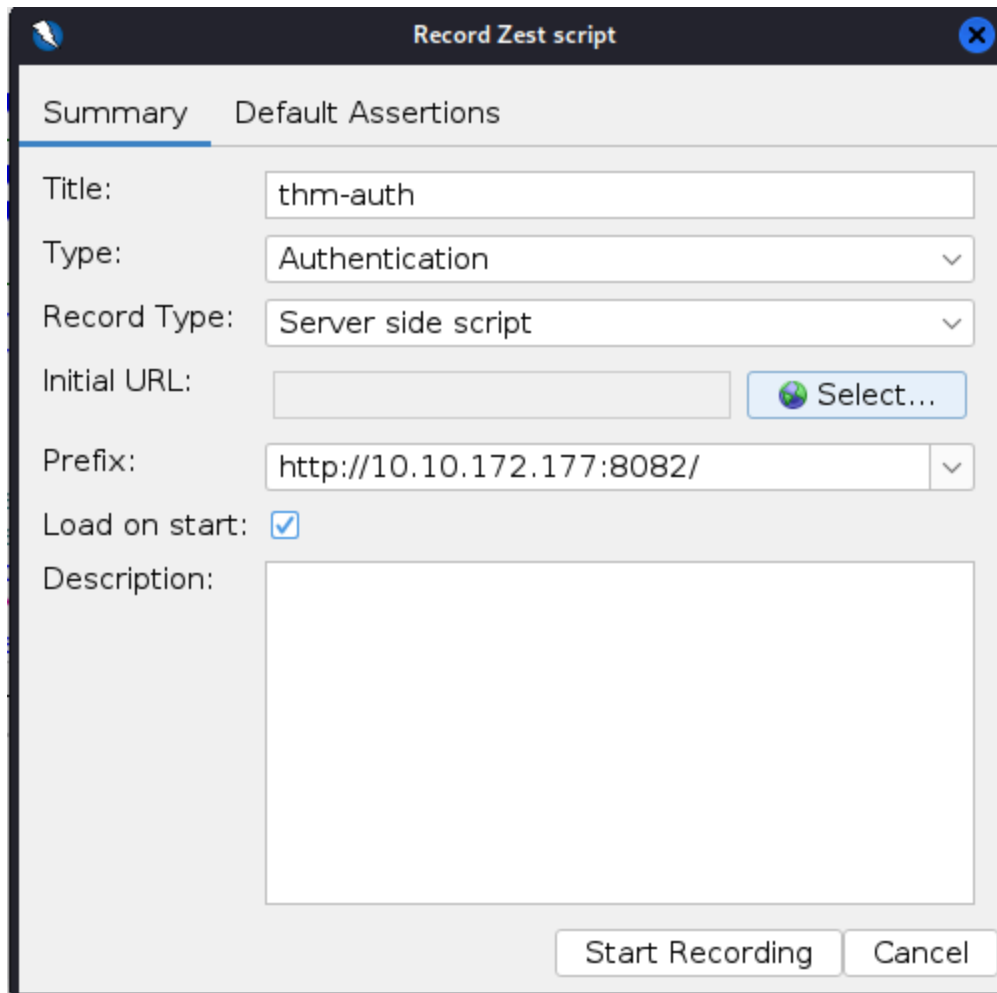
Note: For this task, make sure to disable the ZAP HUD from the toolbar. Some things will not work as expected if it is enabled. You can find the button to do so at the end of the toolbar:



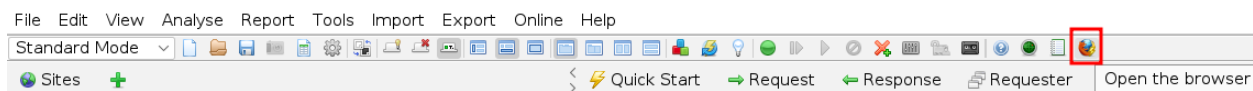
Recording Authentication. To record authentication, we will click the **Record a New ZEST Script** button on the toolbar:



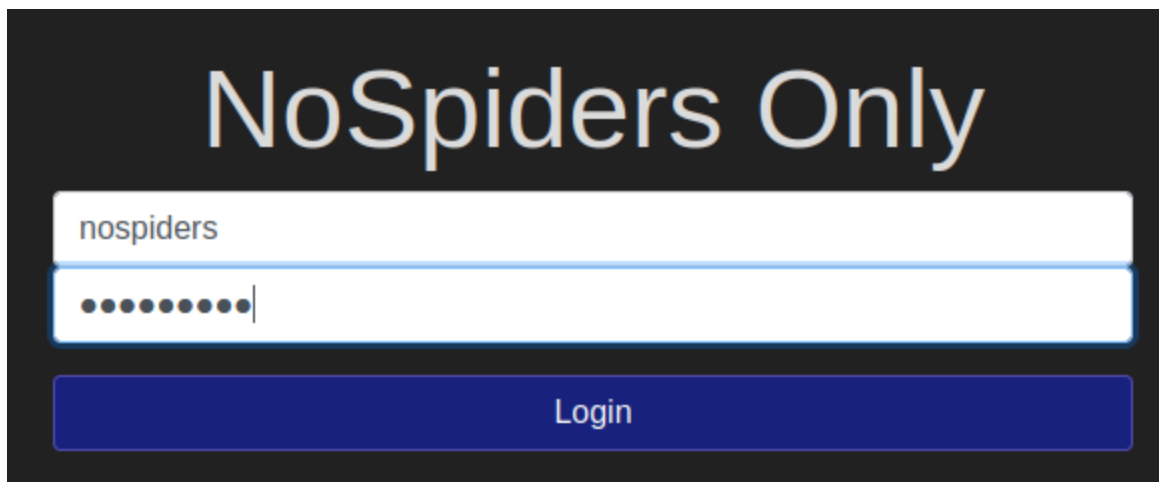
Let's set a title to our script and choose the type **Authentication**. We also want to set the prefix to the base URL of our application, as this will filter out any requests made to other sites:



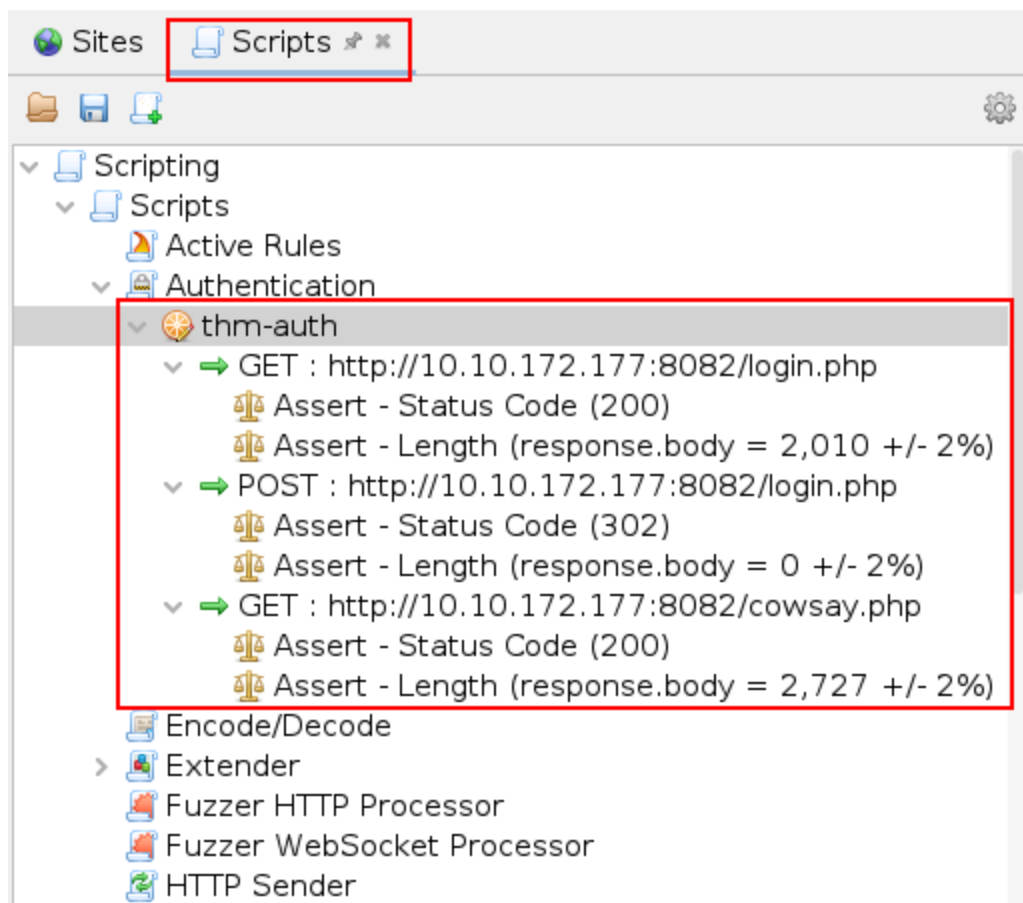
Once we click the **Start Recording** button, every HTTP request that goes through ZAP proxy will be recorded as part of the script. Let's do so and open a browser by clicking the **Open Browser** button on the toolbar:



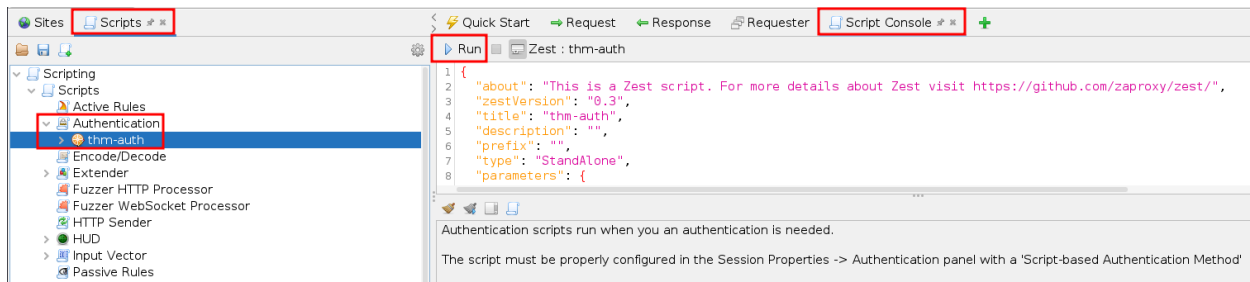
Navigate to http://MACHINE_IP:8082/login.php(opens in new tab) on the newly opened browser. We will log into the website manually so that ZAP can record the process.



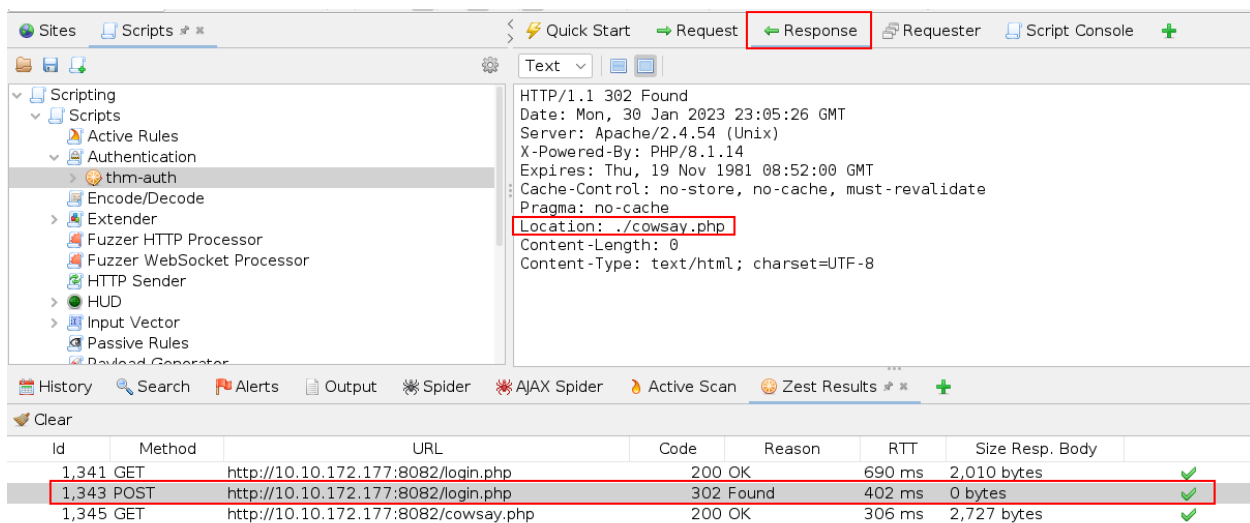
Once we are correctly logged into the application, we will stop recording by clicking the **Record a New ZEST Script** button again. Feel free to close ZAP's browser as well. Some of the captured requests might not be necessary for the authentication process and can be deleted. In this case, these are the only requests that matter:



For each of the requests, ZAP will check the status code and response length to see if they match. If they do, ZAP will assume the login was successful. You can now test the recorded script by pressing the **Run** button on the **Script Console**:



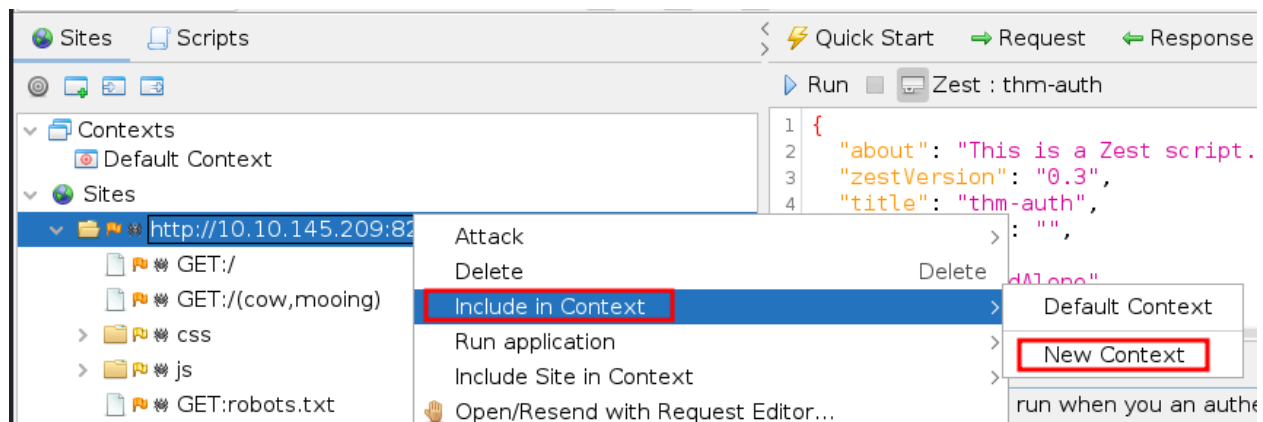
This should allow you to test if you recorded the process correctly by checking the responses to the requests. Our application should respond to the POST request to `login.php` with a redirection to `cowsay.php`:



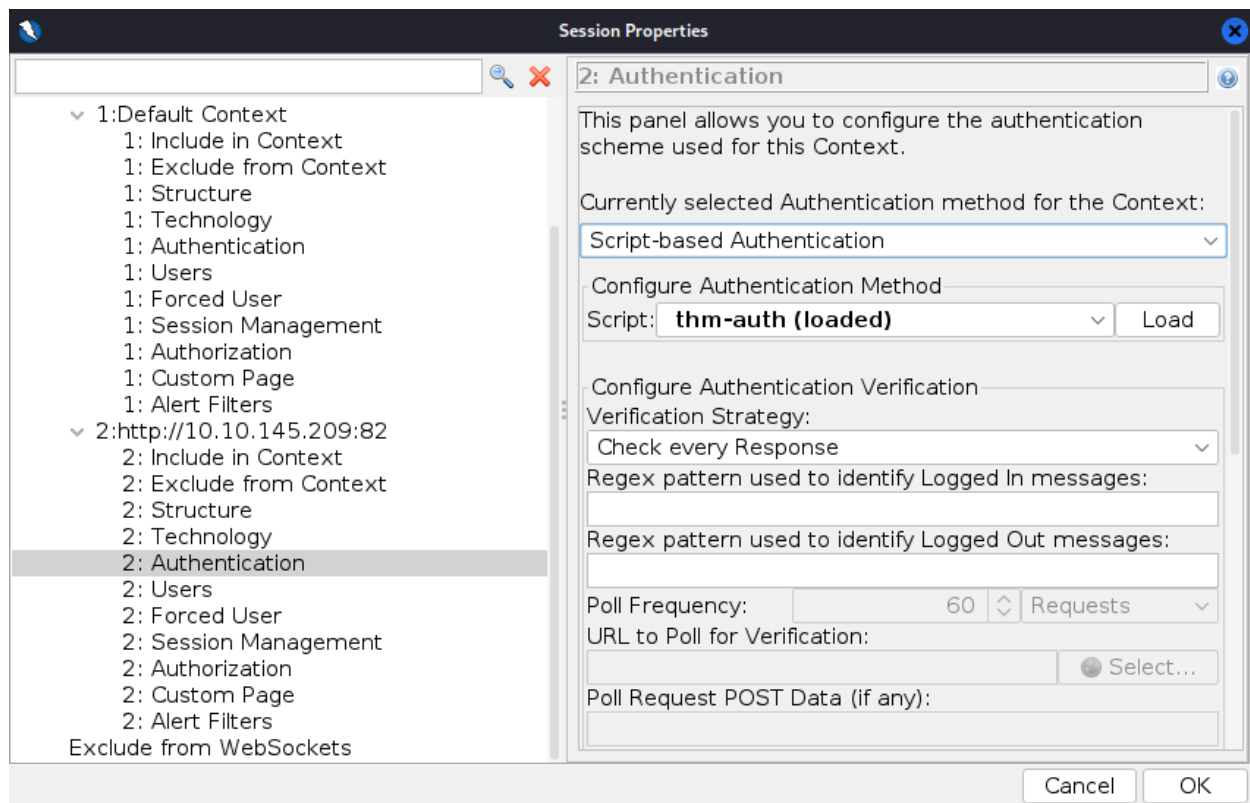
Creating a Context: This is done by creating a **Context**, which is nothing but a way to define a group of URLs. Since this is a simple application, the recorded authentication will apply to the entirety of it. We must therefore define a Context that includes every single URL in the app so that we can link our ZEST script to it.

Note: Depending on your target app, the recorded authentication might grant access to only parts of the application. Think, for example, of an education platform where you have teachers and students. It would be expected that teachers and students each have access to different parts of the application

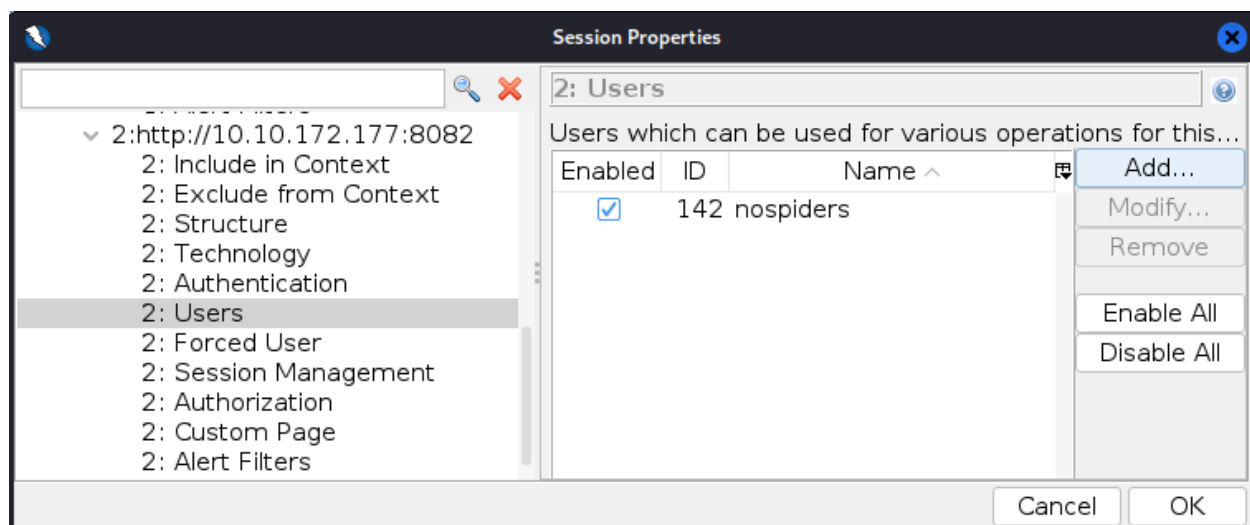
based on their roles. For ZAP to cover both sides of the application, we would need to record two authentication ZEST scripts (one for a student account and one for a teacher) and attach them to two different contexts with the corresponding URLs. To create a new context, go to the **Sites** tab and right-click the base URL of the target application on your **Sites** tab and select **Include in Context** → **New Context**:



This will automatically create a regex that matches any page on the web application. There are many options you can configure under a context. For now, click on authentication, and link your zest script by using **Script-based Authentication**:

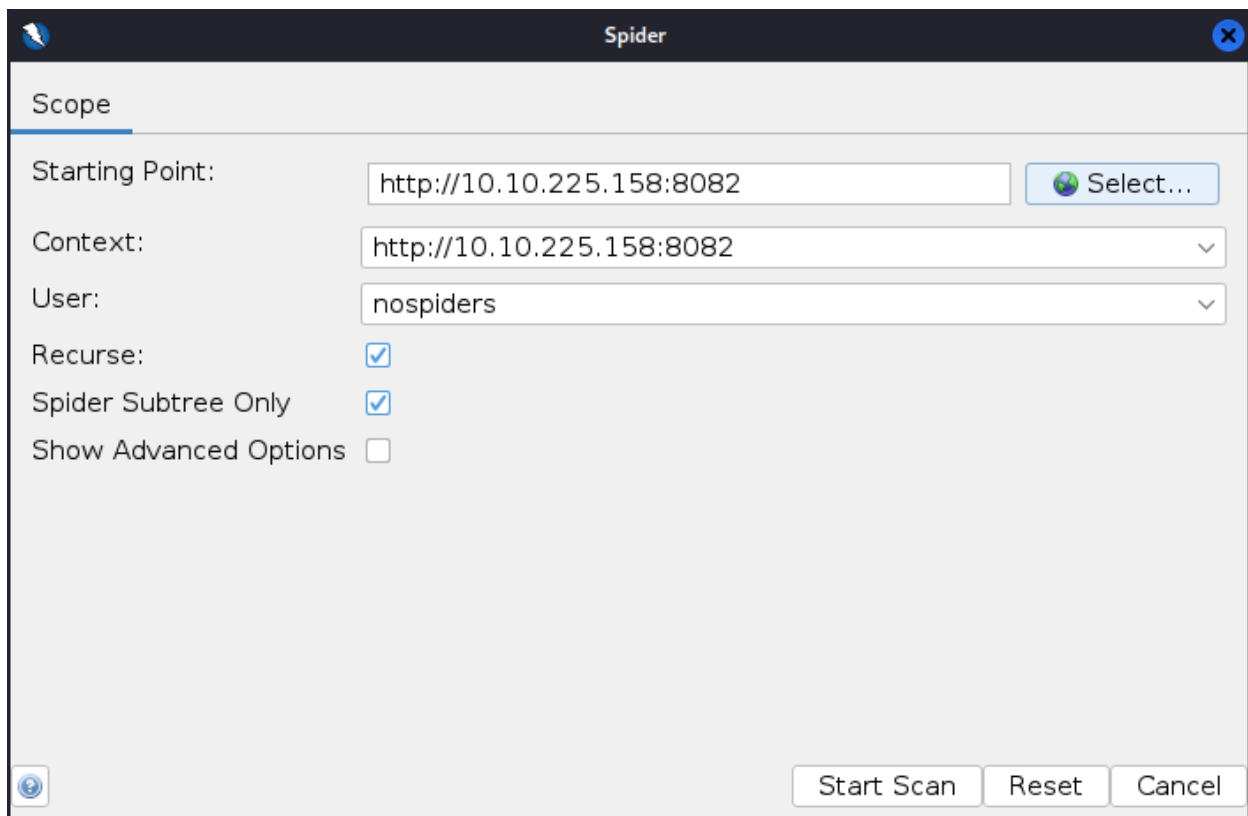


To do authenticated scans, ZAP will also need you to define at least one user in the **Users** section. This won't be used at all in our case, as our ZEST script embeds the user and password we'll use. However, ZAP needs a user to be defined, or it will just skip authentication altogether.

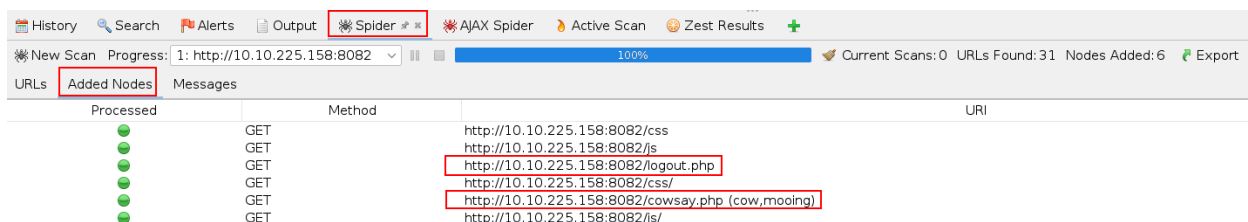


Once this is done, press OK and you should see all of the site's resources are now marked with a target icon, meaning they are now part of the defined Context.

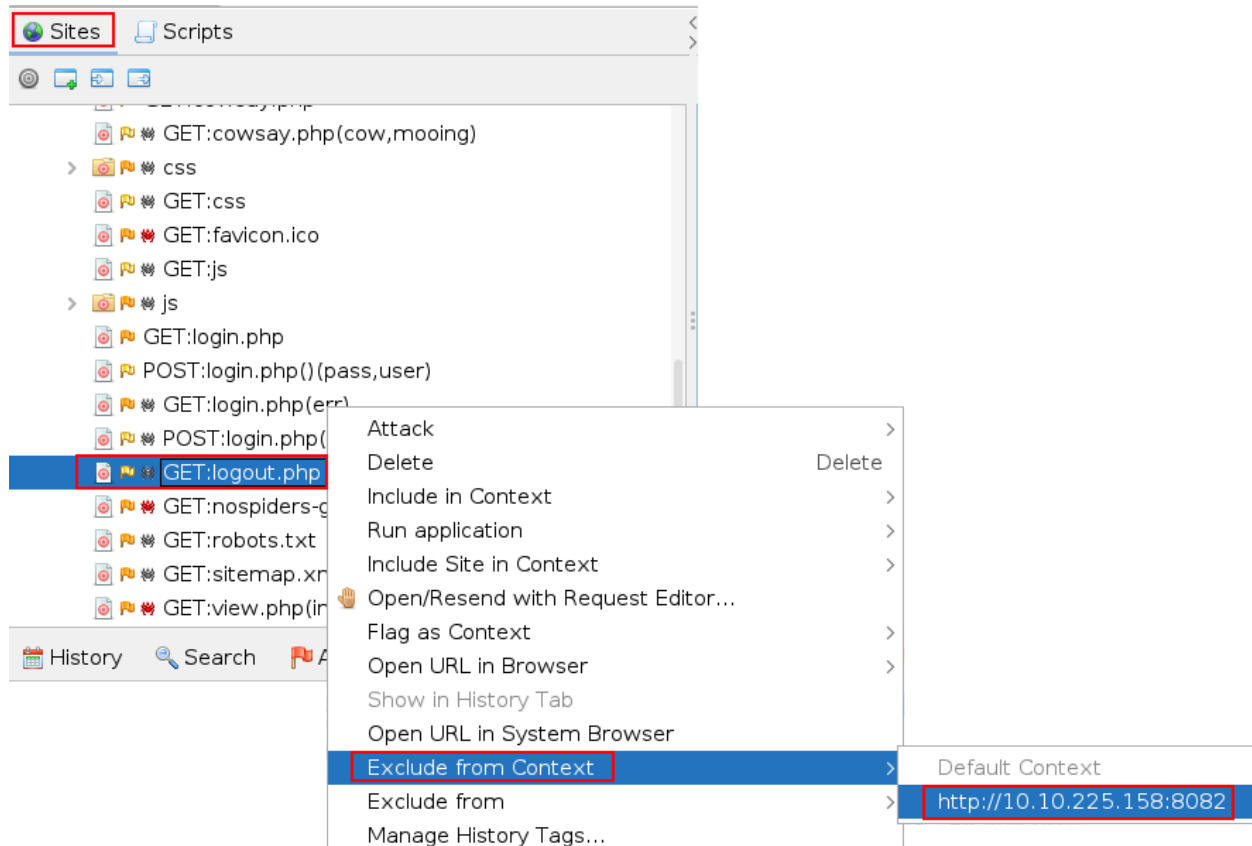
Re-spidering the Application: Now that we have a context set with our authentication script, let's rerun the spider, but setting the correct context and the user we created:



This time, ZAP should be able to find any resources that require authentication and weren't detected before. Let's hit the **Start Scan** button and see what we get. If you want to see if any new resources were discovered during a spidering session, you can always check the **Added Nodes** tab:



ZAP has found a couple of additional PHP scripts. One of such scripts is in charge of logging a user out, and might pose a problem. If ZAP tries to access it, it will log out of its session. To prevent ZAP from logging itself out, we will right-click the logout.php script in the Sites tab and exclude the script from our context:

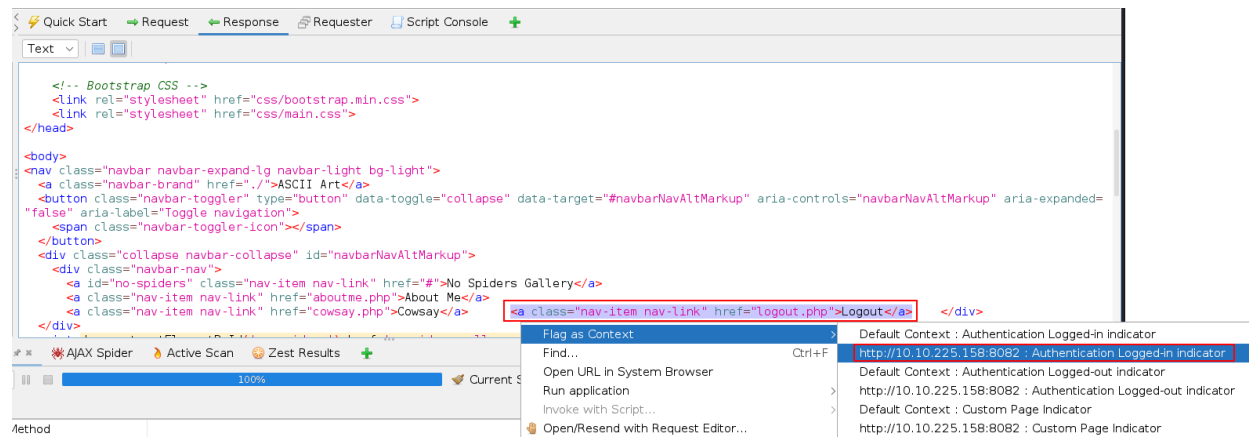


Now ZAP will avoid interacting with the logout script during spidering or scanning. Note that depending on the application, it might be desirable to exclude some other scripts as well. Knowing how the application works is extremely important to get the most out of the scanning process.

In addition to preventing direct logouts, we can specify indicators for ZAP to identify if its session is still active or not, and repeat the login script if necessary. This is easily done by selecting parts of a response that only appear when you are logged in or logged out.

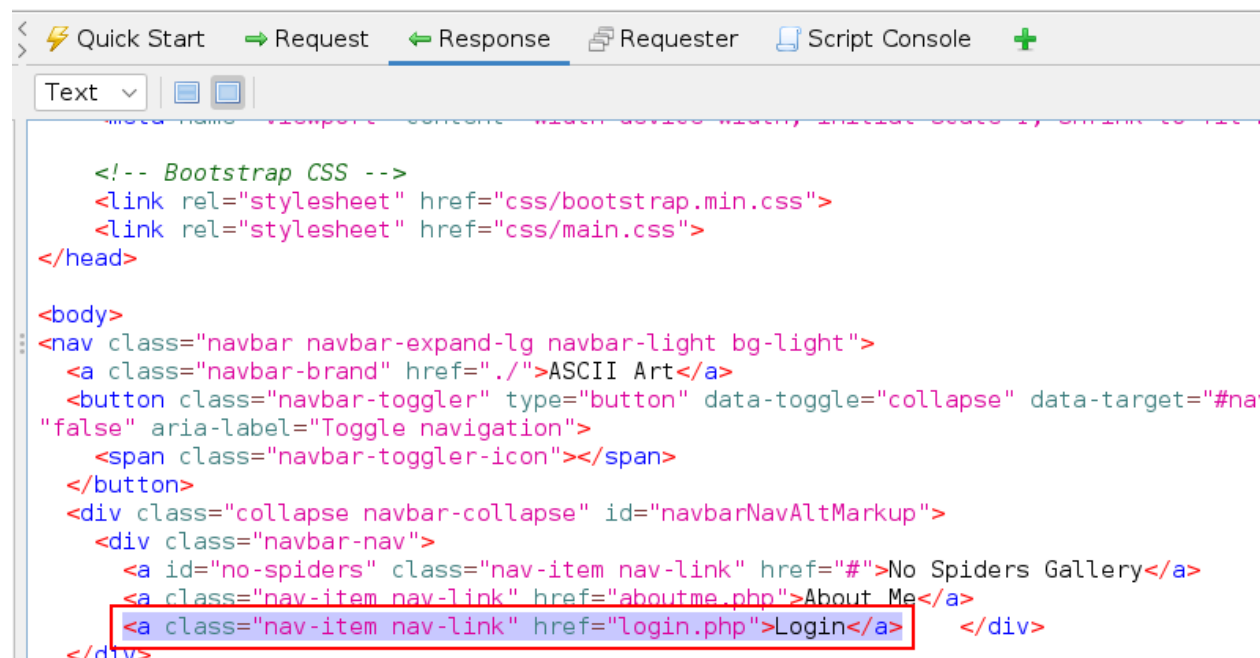
For example, we know that `cowsay.php` can only be accessed if you have successfully logged in, so it might have good indicators we can use for ZAP. Let's select `cowsay.php` from the Sites tab and select the text that corresponds to the

logout link. Right-click the selected text and choose **Flag as Context** → **(Your Context Name): Authentication Logged-in indicator:**



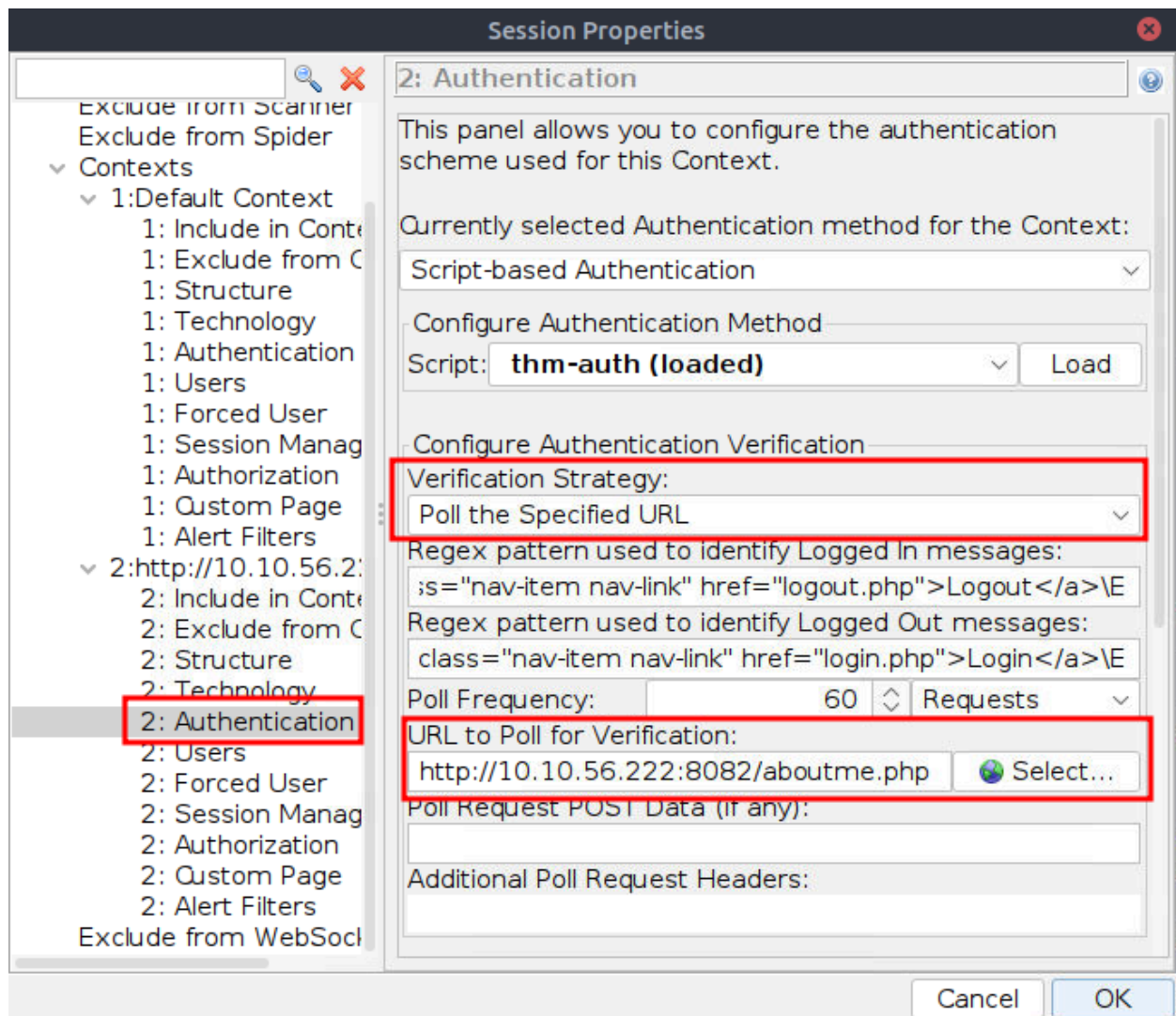
From now on, ZAP will search for the logout link on each response. If it isn't present, it will assume its session needs to be renewed and log back into the application using the ZEST script we recorded.

You can also do the same to select a Logged-out indicator. Since we know the login link should only be visible if you are logged out, we can use it for this purpose by following a similar procedure. Let's grab the link's HTML from the `aboutme.php` script in the Sites tab:

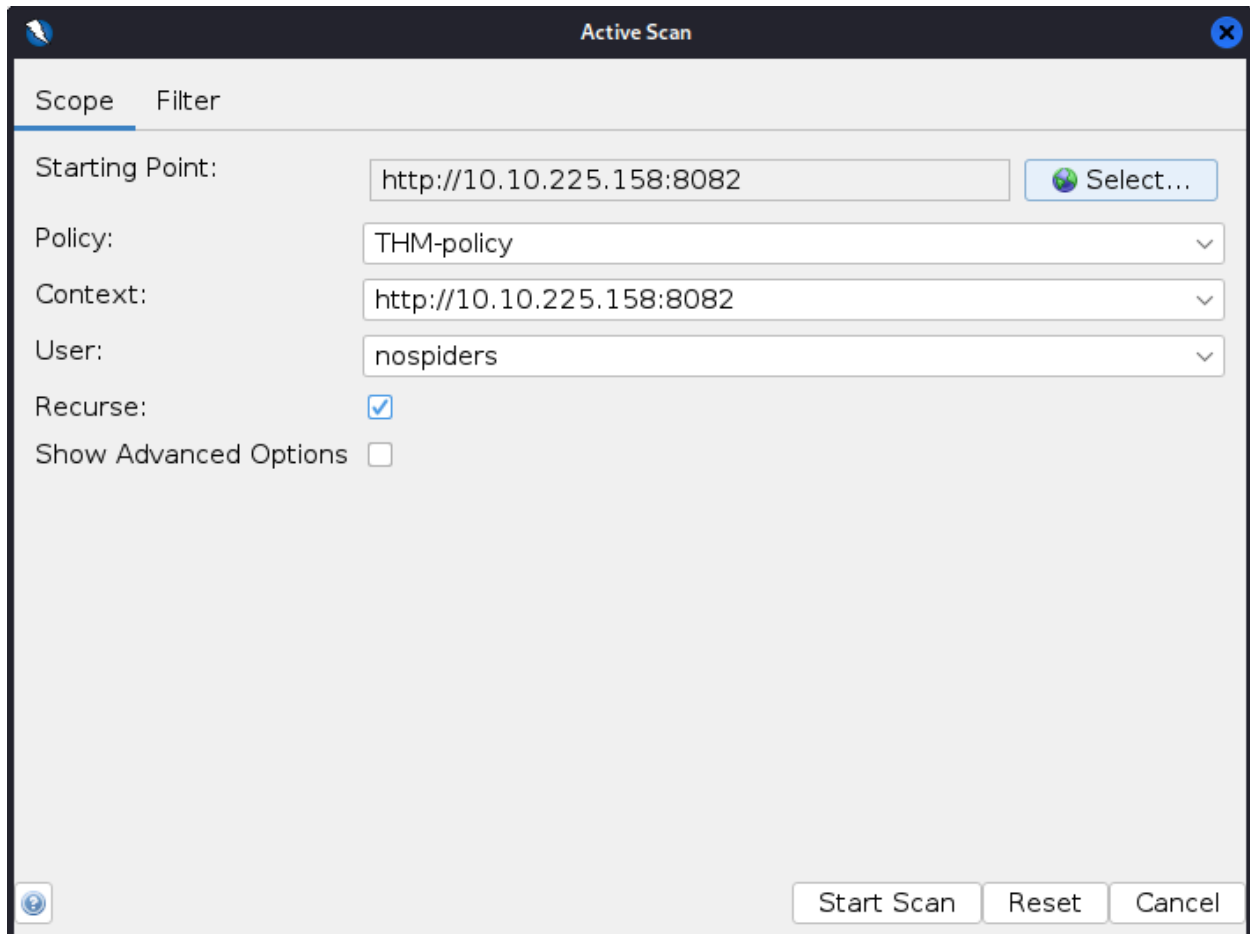


We will right-click the selected text and choose Flag as Context → (Your Context Name): Authentication Logged-out indicator. Note: It may be possible that ZAP has cached the authenticated response for `aboutme.php`. In that case, you will need to resend a request via the built-in browser to that URL to get the correct pattern.

Now that ZAP has patterns to identify if it is logged in correctly, we need to choose a Verification Strategy to tell it how to use the configured patterns correctly. Depending on the website you are checking, you will want to use different strategies and tune them to suit the target application. In this case, we will use the Poll the Specified URL strategy and tell it to poll the `/aboutme.php` resource every 60 requests to check for the logged-in/logged-out patterns. To do this, double-click the Context and change the following settings under Authentication:



Rescanning the Application One Last Time: We are finally set up to run a new scan on the application, knowing that it will now cover the areas of the application that require authentication. Let's go to **Tools → Active Scan**, and select the **Context** and **User** we created:



If you followed the steps correctly, ZAP will now find a new high-risk vulnerability. Be sure to take note of it to respond the questions at the end of the task.

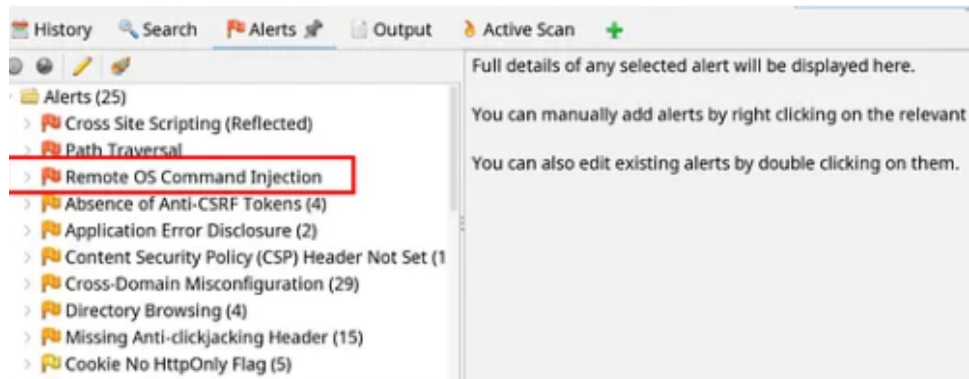
Answer the questions below

Which type of script was used to record the authentication process to our site in ZAP?

ZEST scripts

What additional high-risk vulnerability was found on the site after running the authenticated scan?

Remote OS Command Injection



Task 6 - Checking APIs with ZAP

Nowadays, web applications depend on APIs to perform some or all of their tasks. The DAST process we have shown so far involves being able to spider a traditional web application, but APIs aren't something that can be spidered easily. The endpoints of an API won't normally expose information about other endpoints, making the discovery process harder. Furthermore, even if we get an endpoint's name, we still need to know what parameters can be passed to it. Instead of spidering APIs, we will rely on the development team to give us a precise specification of the available endpoints in an API and all of the parameters we can send to them. This way, we don't need to spend time on the discovery process but can focus on testing its security directly.

API Descriptions: Several standards exist to document APIs, which generally include all the information developers would need to build proper requests and know what to expect as responses for each of the available endpoints. This is also all we need to start testing for security vulnerabilities. ZAP can import APIs defined by **OpenAPI** (formerly **Swagger**), **SOAP** or **GraphQL**. Depending on the API you are testing, you might get one of these formats from the development team.

To give you a better idea of what these files contain, let's take a look at the one published on http://MACHINE_IP:8081/swagger.json (opens in new tab), which follows the OpenAPI 2.0 standard. Inside the file, you will find a section called `paths`, where every single endpoint of the API is listed. For each of the endpoints, a complete list of the available parameters is presented. For example,

here you have the part of the specification on how to interact with `/asciiart/{art_id}`:

```
"/asciiart/{art_id}": {
  "get": {
    "parameters": [
      {
        "description": "art id",
        "in": "path",
        "name": "art_id",
        "required": true,
        "type": "string"
      }
    ],
    "responses": {
      "default": {
        "description": "",
        "schema": {
          "$ref": "#/definitions/ASCIIArt"
        }
      }
    }
  }
}
```

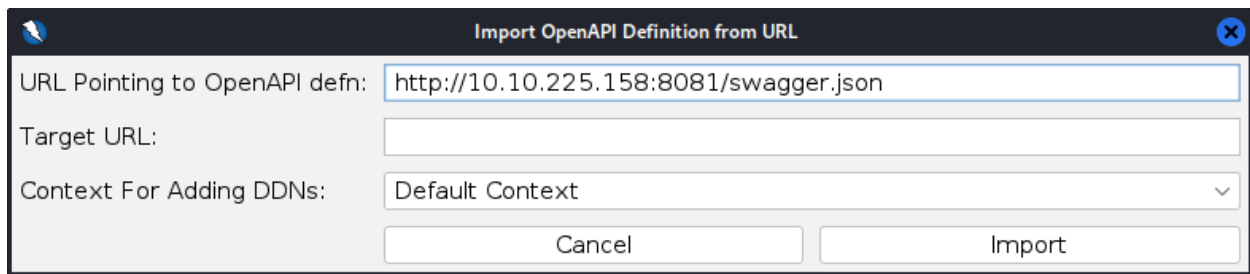
The file indicates that you can do calls to `/asciiart/{art_id}` using the GET method by sending the `art_id` parameter in the URL, which is a string. You can find similar definitions for other paths as `/asciiart/generate` or `/asciiart/add`.

API definition files are expected to be processed by machines, so their format might be hard to read. To make it easier for us to understand them, some APIs will also provide a more friendly UI that we can use to run quick tests and explore the available endpoints. You can go to http://MACHINE_IP:8081/swagger-ui/ ([opens in new tab](#)) to check it.

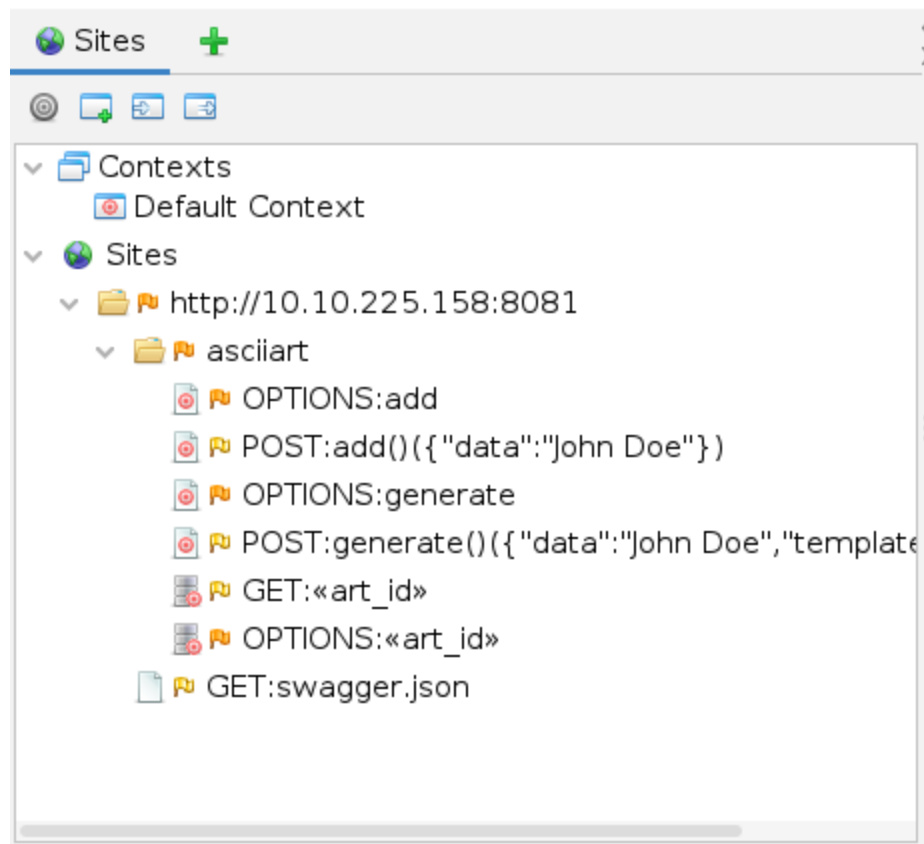
Note: Production applications might hide API definitions to prevent external attackers from knowing how to interact with them. Remember, you can always ask the development team for them if needed (if they exist).

Importing OpenAPI definitions in ZAP: ZAP supports two methods to import API definitions: you can provide an offline file or a URL from where to download them. Since our API exposes its definition file in http://MACHINE_IP:8081/swagger.json ([opens in new tab](#)), let's use the URL

option. Go to **Import** → **Import an OpenAPI definition from a URL**. Specify the corresponding URL and hit the **Import** button.

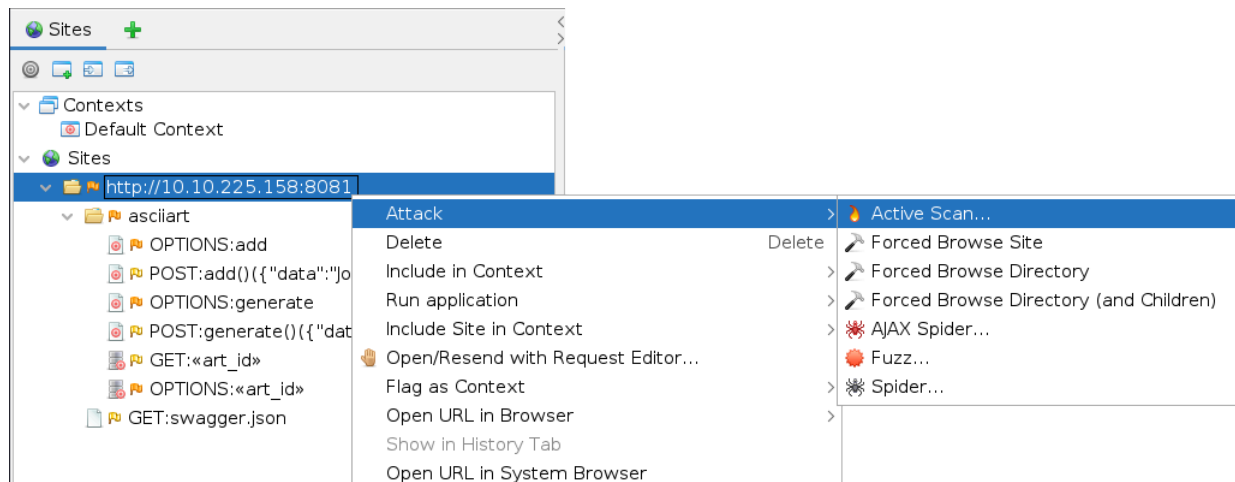


You should now see all of the available endpoints and parameters loaded in the Sites tab.



ZAP will also automatically do passive scanning on the endpoints and report some alerts. After importing the API definition, ZAP will treat the API as any other normal web page.

Scanning the API: Once an API definition is loaded into ZAP, you can use the available tools just as with any other website. To run a scan against the API, right-click its URL and select **Attack → Active Scan**.

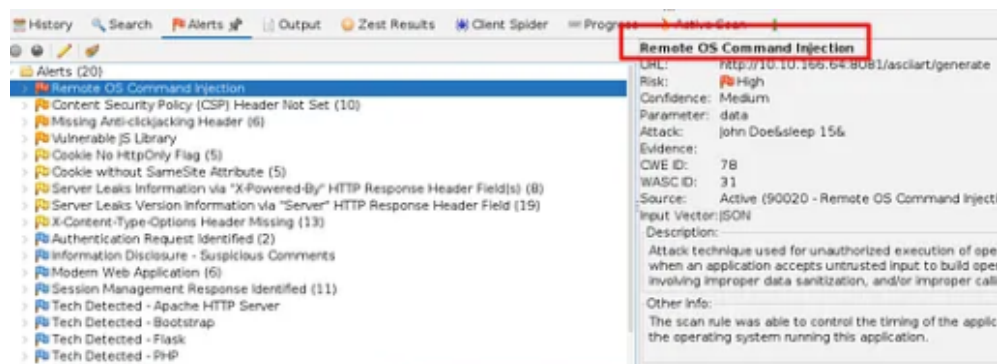


After running the scan, ZAP should have discovered some vulnerabilities in the API. Use the results of the scan to answer the questions at the end of the task.

Answer the questions below

What high-risk vulnerability was found on the /asciiart/generate endpoint?

Remote OS Command Injection



Read the details on the Path Traversal vulnerability detected. Based solely on the information presented by the scanner, would you categorise this finding as a false positive?

yea

Task 7 - Integrating DAST into the development pipeline

Generally, when discussing DAST, people will refer to implementing automated vulnerability scanning into your development pipeline rather than the manual process we have covered. In this way, the term differs from what you would typically call application pentesting. Having DAST added to your development process may sound pretty straightforward, but there are some caveats to it:

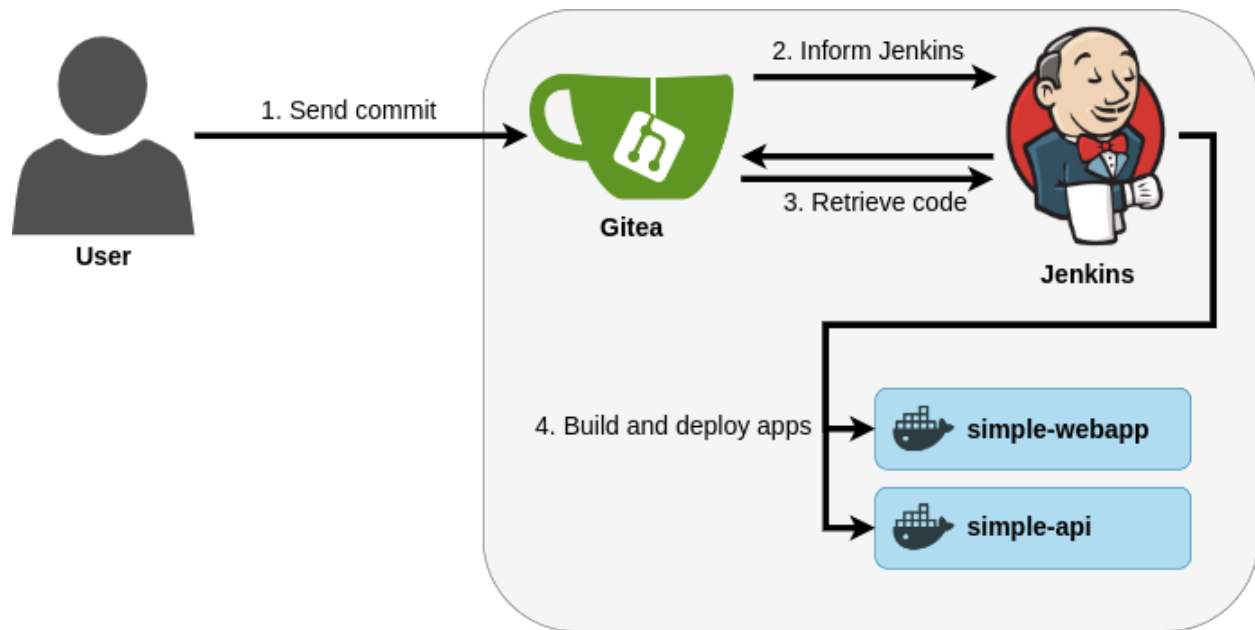
- We must decide at which stages of the development process we will run scans.
- We must decide what will trigger a scan. We can run scans on each commit made to code or on a scheduled basis.
- We must determine the intensity of each scan. Doing a full vulnerability scan on a medium-sized application will require significant time, and we don't want to slow down the development team.

Since no solution fits all scenarios, determining all of this must be done with the help of the development team so that security requirements are met without actually creating significant disturbances to their established processes. In this task, we'll examine how we can quickly implement DAST directly into the development pipeline to provide early visibility of vulnerabilities. The scans will run automatically on each commit and will test for a subset of vulnerabilities only to avoid introducing long delays in the pipeline.

Reviewing our CI/CD Pipeline: Our current machine implements the entire development pipeline for the web application and API you scanned previously. The following two components are in charge of doing that:

- A Gitea repository is running on http://MACHINE_IP:3000/(opens in new tab), where the code for both applications is stored.
- A Jenkins instance that runs the CI/CD pipeline. You can access Jenkins at http://MACHINE_IP:8080/(opens in new tab).

Every time a commit is made in Gitea, Jenkins will take the code from the repository and compile it into a Docker instance. These Docker instances are what you have scanned so far.



Running Automated Scans With zap2docker: To integrate ZAP in our pipeline, we will use **zap2docker**, a dockerized version of ZAP proxy built with automation as its primary purpose. The full documentation for zap2docker can be found [here\(opens in new tab\)](#). **Note:** The commands in this section are provided for reference only. You don't need to run them manually, as we will have Jenkins do it for us later. To install zap2docker, you can pull it from Docker Hub using the following command:

```
docker pull owasp/zap2docker-stable
```

You can run ZAP from the docker instance using one of the packaged scan scripts, which will allow you to run one of the following scan profiles: **Baseline Scan:** ZAP will spider the target website for a maximum time of 1 minute. No active scan will be performed. You can optionally run an AJAX spider if desired.

```
docker run -t owasp/zap2docker-stable zap-baseline.py -t https://www.example.com
```

Full Scan: ZAP will run a spider with no time limits, followed by an active scan.

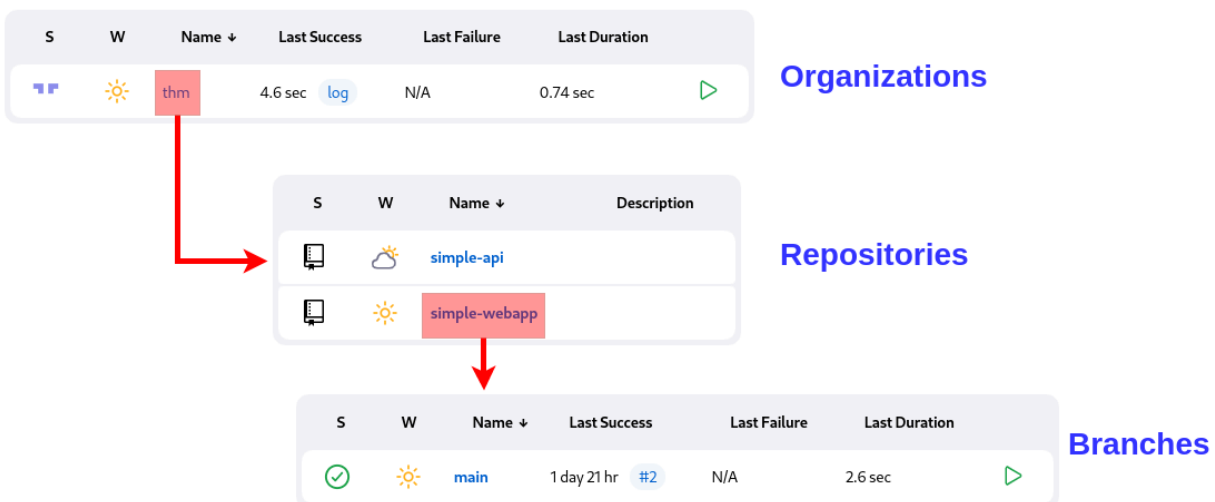
```
docker run -t owasp/zap2docker-stable zap-full-scan.py -t https://www.example.com
```

API Scan: ZAP will perform an active scan against an API. You will need to provide a URL to an API description file (OpenAPI, GraphQL or SOAP).

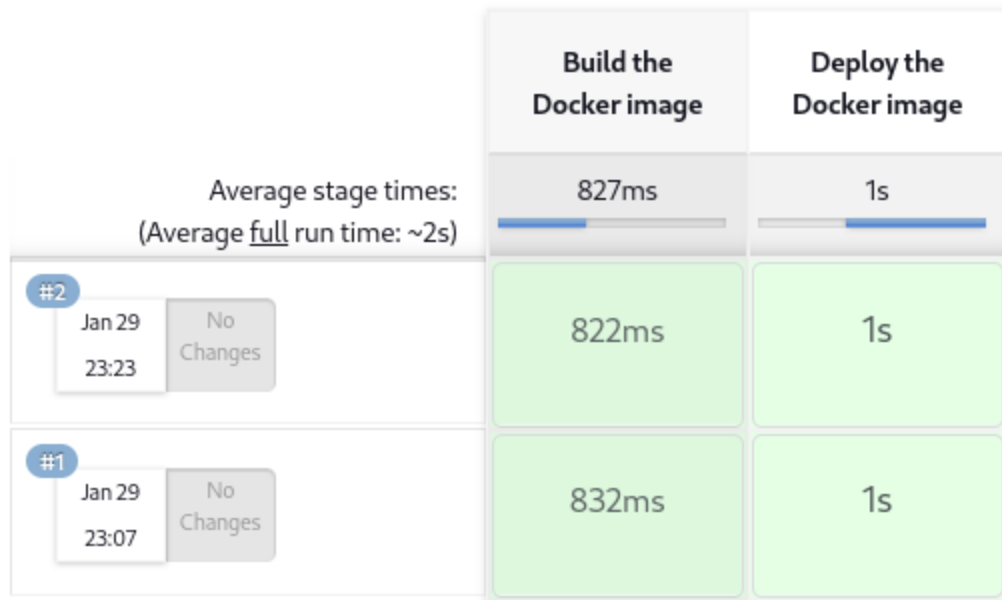
```
docker run -t owasp/zap2docker-stable zap-api-scan.py -t https://www.example.com/swagger.json -f openapi
```

In any of the scans, the results for passive scans will be limited to 10 alerts. In addition to the base commands, you can add the `-j` switch to perform an AJAX scan on the baseline and full scans.

Integrating ZAP Into the Pipeline: The environment you have access to already has a zap2docker instance ready to use. All we need to do is to tell Jenkins to run zap2docker with the options we need, and that will be it. Before going into that, let's explore what we have in Jenkins. When logging into Jenkins, you will find an **organisation** called `thm`, which contains the two **repositories** for the web application and API. For each of the repositories, you can see the available **branches**.



You will have a single branch in both projects called `main`. If you click on the branch, you will be able to see the builds for it. The `simple-webapp` project, for example, has been built two times already:



For each build, we can see there are two stages configured:

- **Build the Docker image:** This stage will build the Docker container based on the Dockerfile on each project.
- **Deploy the Docker image:** This stage will deploy the Docker container.

Both of these stages are defined in the **Jenkinsfile** in each repository. If we go to Gitea and open the [Jenkinsfile](#)(opens in new tab) on the simple-webapp repository, we can see the specific commands run on each of the stages:

```
stage ('Build the Docker image') {
  sh "echo building the image..."
  sh "docker build --tag simple-webapp:latest ."
  sh "echo building image complete."
}

stage ('Deploy the Docker image') {
  sh "echo Deploying the container..."
  sh "docker rm -f simple-webapp"
  sh "docker run -d -p 8082:80 --name simple-webapp simple-webapp:latest "
  sh "echo Container successfully deployed."
}
```

In the second stage, you can see how the web application you scanned manually is deployed to port 8082, for example. If we want to incorporate zap2docker into our pipeline, we can add a third stage that runs the scan for us after building the container. For your convenience, the stage is already defined at the end of the Jenkinsfile, and you can uncomment it to get it running. Feel free to use `git` to send a commit to the repository or use Gitea's built-in editor if you find it easier:



```
1 node{
2
3   git branch: "main", url: "http://172.17.0.1:3000/thm/simple-webapp"
4
5   stage ('Build the Docker image') {
6     sh "echo building the image..."
7     sh "docker build --tag simple-webapp:latest ."
8   }
9 }
```

Notice that we are running the **Baseline Scan** only, as this scan will be run for each commit made to the repository, so it isn't sensible to delay the pipeline with a Full Scan that might take hours to finish. This scan intends to find low-hanging fruit only. This means you won't see any SQL injection, Cross Site Scripting, or anything else that requires an active scan. Since Jenkins will catch any changes in the repository, it will start the building process immediately after you edit the file. You can switch back to the simple-webapp project in Jenkins to see the progress live:



You can see that the current build now adds the `Scan with OWASP ZAP` stage. The scan will take around 5 minutes. Once finished, the build will fail, as ZAP will find some vulnerabilities in the app. To check the report, open the build and go to the **Workspaces** section on the menu:

Jenkins

Search (CTRL+K) TryHackMe log out

Dashboard > thm > simple-webapp > main > #3

Status

</> Changes

Console Output

Edit Build Information

Delete build '#3'

Git Build Data

Replay

Pipeline Steps

Workspaces

Previous Build

Build #3 (Feb 1, 2023, 2:35:52 AM)

Keep this build forever

Started 51 min ago

Took 4 min 31 sec

Add description

Changes

1. Update 'Jenkinsfile' (details)

Push event to branch main at 2:35:46 AM on Feb 1, 2023

Revision: 04ee54dbd82bc556508e8078c35ccb97a52c69ff

Repository: <http://172.17.0.1:3000/thm/simple-webapp>

refs/remotes/origin/main

Here you can explore the project files, and ZAP will put its reports in the zap-reports folder. Download and examine the report to answer the question at the end of the task.

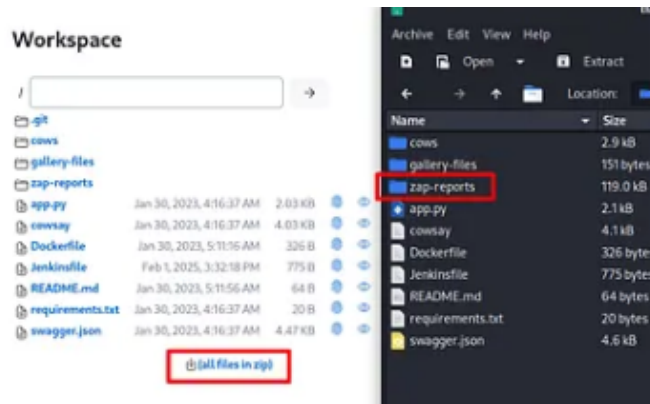
Note: You should right-click and save the report before opening it. Opening it directly from Jenkins will break the formatting.

Repeat the same process to enable ZAP on the simple-api repository and get the corresponding report. Use the report to answer the questions at the end of this task.

Answer the questions below

Download the ZAP report for the simple-webapp repository. How many medium-risk vulnerabilities were found?

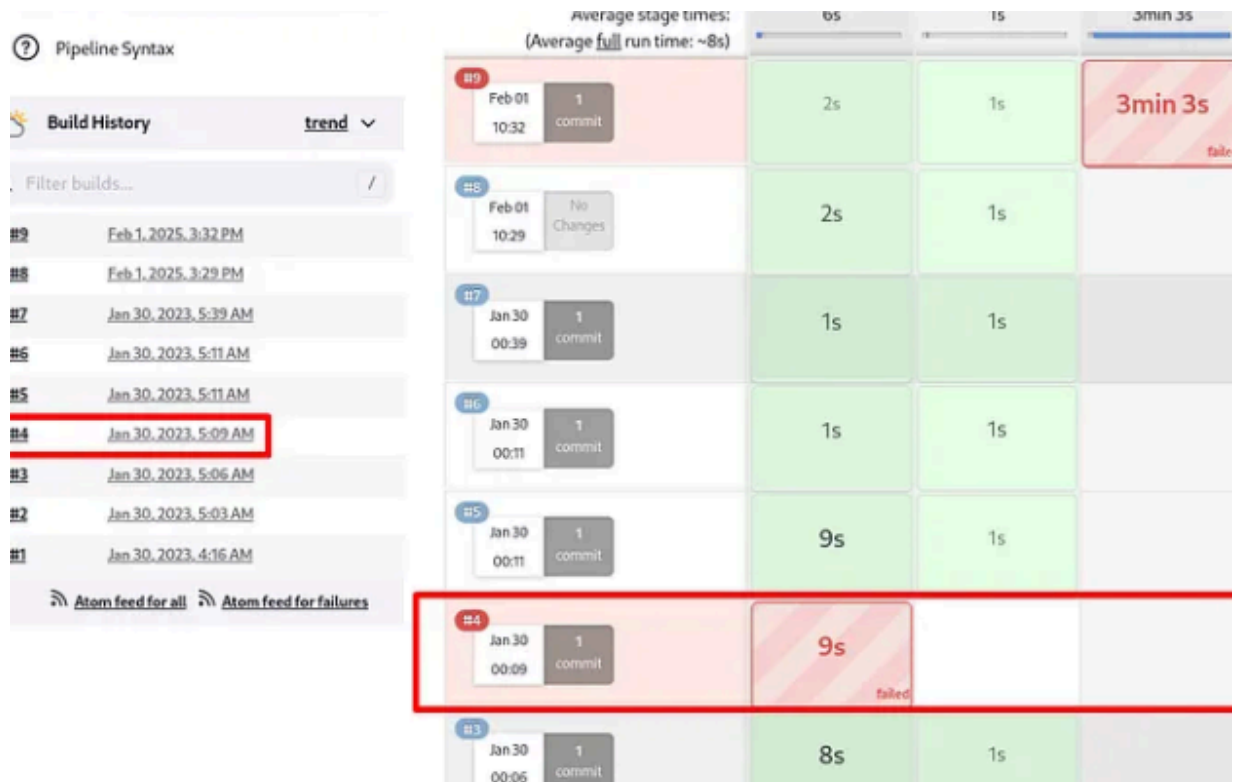
3



Risk Level	Number of Instances
High	1
Medium	3
Low	1
Low	7
Low	66
Low	4
Informational	63

Check the main branch of the simple-api repository on Jenkins. One of the builds failed during the Build the Docker image step. What is the number of the pre-existing failed build? .

4



Download the ZAP report for the simple-api repository. What high-risk vulnerability was found?

Remote OS Command Injection

Name	Risk Level	Number of Instances
Remote OS Command Injection	High	1
Content Security Policy (CSP) Header Not Set	Medium	3
A Server Error response code was returned by the server	Low	1
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	7
Unexpected Content-Type was returned	Low	66
X-Content-Type-Options Header Missing	Low	4
A Client Error response code was returned by the server	Informational	63

Task 8 - Conclusion

In this room, we have covered the basics of how DAST works and introduced ZAP proxy as a tool that can perform DAST in manual and automated ways. DAST is only one of many ways to check for applications' vulnerabilities and should be used in tandem with other types of testing like SAST, SCA, penetration tests and others to guarantee a reasonable security level for our applications. As with any other technique, DAST won't be a silver bullet solution but will contribute

to an application's overall security throughout the software development lifecycle.

DAST (Dynamic Application Security Testing) is a type of security testing that analyzes a running application to find vulnerabilities by simulating real-world attacks. Unlike SAST, it does **not require access to source code**.

What DAST Does: DAST tools interact with a **live application (frontend + backend)** and try to identify security weaknesses such as:

- SQL Injection
- Cross-Site Scripting (XSS)
- Authentication bypass
- Session management issues
- Insecure cookies
- Command injection
- Open redirects
- Security misconfigurations

How DAST Works

1. The application is deployed and running (staging or production-like environment).
2. The DAST tool crawls the application like an attacker.
3. It sends malicious inputs (payloads) to input fields, URLs, APIs, etc.
4. It observes responses for signs of vulnerabilities.
5. It generates a report with findings and severity levels.

Key Characteristics

- 🔍 **Black-box testing** (no access to source code)
- ⚙️ Tests **running applications**
- 🌐 Simulates real attacker behavior
- 📡 Can test APIs, web apps, and services

Benefits of DAST

- Finds runtime and environment-related vulnerabilities
- No need for source code access
- Detects issues missed by SAST
- Useful for staging and production testing
- Helps validate real exploitability

Limitations of DAST

- ✗ Cannot pinpoint exact vulnerable code line
- ✗ Requires a running application
- ✗ Slower than SAST
- ✗ Limited code coverage (depends on crawling)
- ✗ May miss deep logic flaws

Popular DAST Tools

- OWASP (OWASP ZAP) – open-source web application scanner
- Burp Suite (PortSwigger) – widely used professional web security testing tool
- Acunetix – automated web vulnerability scanner
- Nessus – broader vulnerability scanner (includes web testing capabilities)
- Qualys Web Application Scanning – cloud-based DAST solution

DAST vs SAST

Feature	DAST	SAST
Testing type	Runtime (dynamic)	Code analysis (static)
Code required	✗ No	✓ Yes
Approach	Black-box	White-box
Finds runtime issues	✓ Yes	✗ No
Best stage	Testing / staging	Development

DAST is security testing that attacks a running application like a hacker would, to find vulnerabilities before attackers do.